Develop a Digital Technologies Outcome Template

the Assessment and Workbook for more detailed instructions.

Task 1: Set-up Project Management (A)

Project Overview & Requirements

The purpose of project is to develop a spaced-repetition web application ("NCEA Review Navigator") that helps students manage their revision workload, track their confidence in different topics, and reduce study overwhelm by automatically scheduling when they need to review material.

This project aligns with SDG 4: Quality Education, by providing a digital tool that promotes effective, lifelong study habits and equitable access to better learning techniques (spaced repetition) rather than last-minute cramming.

The end users are NCEA students (Years 12-13) in New Zealand who are stressed, juggling multiple subjects, and struggling to organise their revision schedules effectively.

Core Requirements:

1. Users must be able to create secure accounts where their personal study data is strictly isolated from other users.

2. The system must accurately calculate and schedule next-review dates based on spaced repetition schedule.

3. Users must be able to pause/mute subjects during busy internal assessment weeks.

4. The UI must be clean, uncluttered, and easy to navigate.

Project management tools

What project management tools are you using?
I use trello and google sheet progress date tracker as my project management tools.

Why are these tools appropriate for the development of your outcome?
For managing my project, I am using Trello as my main planning tool.I used an iterative, MVP-first approach managed on a Kanban board. I chose it because each component needed to be built, trialled, tested and refined before the next depended on it and the scheduler couldn't be trialled until the database and log-review flow existed. I use it to break my project into clear task like hp design, database setup, review logic, and testing. I move tasks through stages like “to do”, “doing”, and “done” . This helps me visually track progress and makes sure I don't forget smaller steps inside the project. I also use a google sheet milestone tracker as the fixed-deadline. This is important to me because my outcome includes a lot of complicated components and I need a strict timeline/due date to make sure that I actually finished developing my outcome on time. These tools are mainly for organisation and keeping the project structured so I always know what stage I am at.

Project management techniques

How are you using/applying the project management tools you have selected?
Consider, how you might prevent account for technical faults or loss of files?
You might also want to account for sickness or any events that might de-rail your milestones?

Examples of techniques include:
- saving backup copies with a logical file naming system
- adjusting key actions and tasks where appropriate

Which project management techniques are you using?

The techniques that I'm using are prioritising tasks, flexible schedule and progress monitoring. (Agile incremental development)

Explain why these techniques are appropriate for the development of your outcome.
I am using Agile incremental development alongside Kanban progress tracking and flexible scheduling. Using an incremental approach means I focus on building and integrating one complete component at a time, like setting up the SQLite database, writing the Flask backend logic, and connecting the frontend of an entire component (e.g. auth screen) before moving on to the next component. Testing these connections step-by-step can prevent major integration issues late in the project where the separate systems might fail to talk to each other. I pair this with a Kanban board to visually monitor my progress and track which features are finished or still need work, keeping the overall build organised. Finally, I also maintain a flexible schedule that allows me to adjust my timeline whenever complex backend features or debugging take longer than planned, ensuring I don't rush critical code just to hit a rigid deadline.

Version control

What version control system are you using to plan the development of your outcome? What tools/techniques are you using?
The version control system I use is Github and local backup copies.

Explain why this version control system/tools/techniques are appropriate for the development of your outcome.

For version control, I use GitHub as my main system to track changes in my code. Every time I make a good change, I commit a new version so I can always go back if something breaks or stops working. To accompany github, I also use source tree to re-analyse and see what changes have been made and comment if it's approved/suitable. Because sometimes I may have changed something and I may want to look at it again to see whether that change I have made, and perhaps be able to see the difference/the previous so if I want to have that code back, I can use the source tree to see the old code.

Alongside GitHub, I also use a simple file naming system as a backup, like: homepage_v1, homepage_v2, backend_test_v1

This gives me extra safety in case I make a mistake or need to compare different versions (i can go back).

I also keep local backup copies of my project files. This is because it means my work is not only stored online but also saved directly on my device.Version control helps me avoid losing work and lets me experiment more safely because I know I can always go back to a working version.

Implications:

Before I started building NCEA Review Navigator, I identified four implications I expected to be relevant, based on the fact that it stores real student data (names, emails and passwords) and is aimed at supporting student wellbeing and education under exam stress:

Privacy: because the web stores personal login details and subjects data, I planned to hash passwords rather than store them as plain text, and to make sure one student's data could never be visible to another student's account.

Intellectual property: because I expected to use tutorials and open-source examples to learn techniques I hadn't used before (ES Modules, JWT authentication, spaced repetition scheduling), I planned to use these only as learning references, write my own schema, branding and UI from scratch, and use a properly licensed icon set rather than unattributed images.

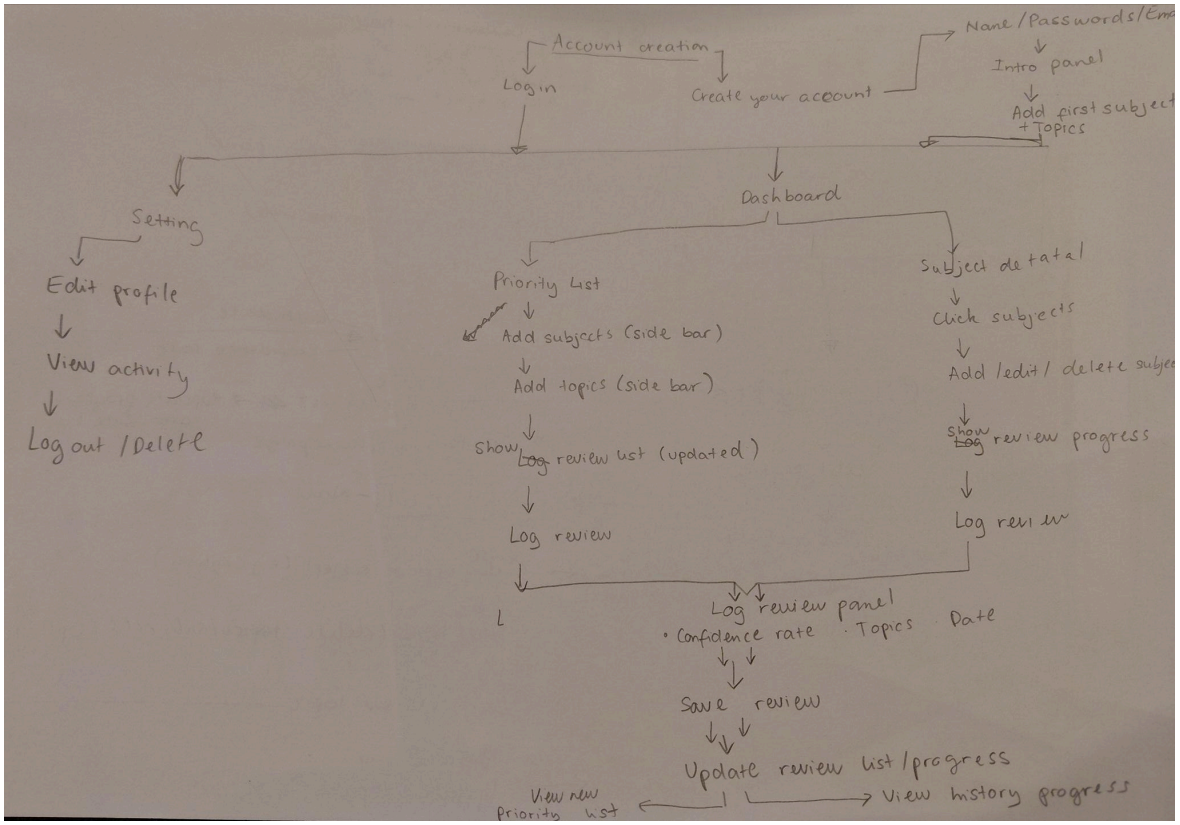
Usability and functionality: because the target users are stressed, time-poor students, I planned to keep the interface low-friction and to test core actions under real working conditions, not just in a static prototype.
End-user consideration:becaúe my end users are NCEA students under exam pressure, I planned to design the scheduling logic around not overwhelming them with too much material at once, and to make the outcome feel supportive rather than purely functional.

I returned to and expanded on each of these implications in Task 5, once I had real testing, trialling and user-feedback evidence of how they actually played out during development.

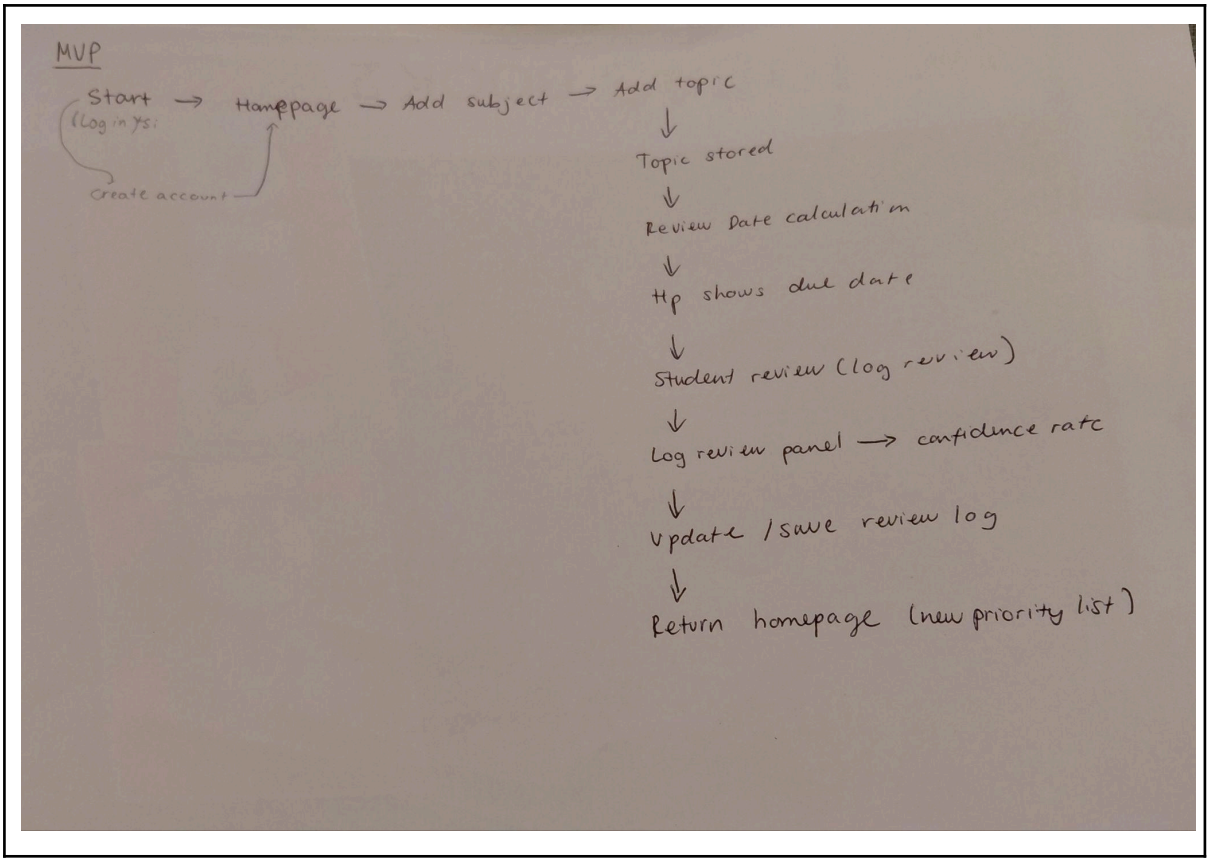
Task 2: Manage Components (A-M)

Components refer to different sections or pieces of your outcome. To make building your outcome more manageable it is important to break it down into smaller chunks.

Flow chart for your full outcome Create a flow chart of what your outcome should do and act (with all the bells and whistles).



Your MVP
Create the flowchart for your MVP(what does the striped back version of your outcome look like?).



This MVP focuses on building a simple spaced repetition tool that gives students daily study recommendations based on their learning pace. At its core, the web lets users create an account to save their personal study data and input the specific subjects or topics they want to track. Once topics are added, the scheduling engine calculates when each item is due and displays those upcoming reviews directly on the user's homepage. After completing a study session, the student logs what they reviewed in the review panel, which instantly prompts the engine to calculate and update the next optimal review date.

Components

Create a list of the components required to build the MVP version of your outcome.
For each component, record the objectives, assets and rules.
The Objectives: What does each component need to do/ look like?
The Assets: What do you need to create to make this component?
The Rules: Are there any rules this component needs to adhere to?

Note: Instead of including them here you might choose to add them directly to your planning tool.

During my initial planning in Task 2 and on Trello, I chose to group the scheduler directly under the "Dashboard" component card rather than creating it as a completely separate feature. Since the main purpose of the dashboard is to give students an instant summary of their daily workload, separating the scheduler onto its own page or board would have created unnecessary planning friction. Planning them together allowed me to design the frontend dashboard layout and the backend scheduling logic as one connected, vertical feature. But because the scheduling itself is quite heavy loaded, i document it as a seperate component in this doc so I discuss more detailed analysis relate to it.

Component 1: Authentication (log in system)
Objectives
Let user create account and return with saved data
Assets
- Log in page - Sign up form - User table (SQ lite) - Password storage system
Rules
User must be logged in to access dashboard and each user only sees their own data

Component 2: Onboarding
Objectives
Allow student to add/input what they’re studying (subjects/topics)
Assets
- Add subject form - Add topic form - Database tables of subjects & topics

Rules
- Topic must have a “last reviewed date” - Each topic must assigned in a subject

Component 3: Dashboard (homepage)
Objectives
Shows student what they need to do today without thinking
Assets
- Cards layout (UI) - “Due today” sections - Subjects progress sidebar
Rules
- Prioritise overdue topics first - Updates in real time - Minimal clutter (no overload)

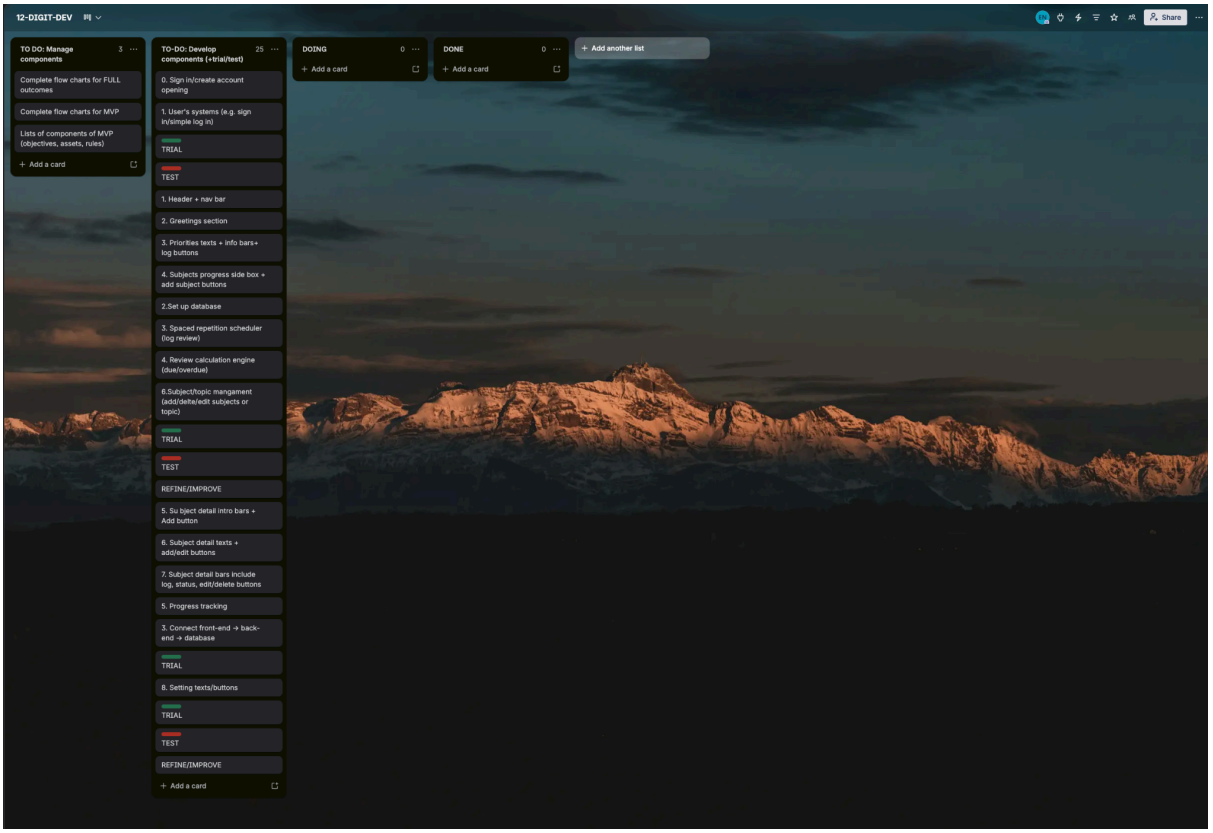
Component 4: Spaced Repetition Scheduler
Objectives
Decide when a topic should come back.
Assets
- Simple interval logic (e.g. 1 day → 3 days → 7 days → 14 days) - Database field: next_review_date
Rules

- Must always set a next review date after completion - Must prioritise overdue topics first - a maximum of 5 review cards per day overall, of which at most 1 may be a brand-new topic per subject; overdue topics are always ranked first.

Component 5: Subject detail page
Objectives
Shows student specific topic progresses + evidence of logging
Assets
- Progress bar (% completed - “Due today” sections - Internal assessment mode button - All topics belong to subject + confidence rating log
Rules
- Updates in real time - Minimal clutter with clear organisation of topics arranged in due date - Easy to read + understand layout where users can easily navigate

Incorporate these components into your project management tool.

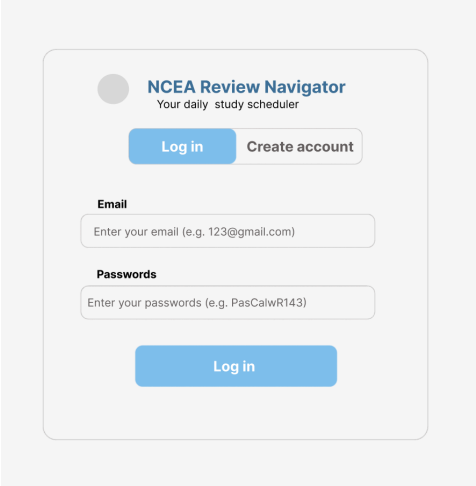
1	**Development PROJECT MILESTONE TRACKER**					
2						
3	STATUS KEY: Completed Revised/Late Rescheduled Upcoming					
4	#	Milestone / Task	Deadline	Time took to complete	Time completed	Status
5	▶ SETUP					
6	1	Set up git + project folder	22/06/2026			
7	2	Download python + technology needed	24/06			
8	3	Set up basic html + css+ js files	25/06			
9	▶ COMPONENT DEVELOPMENT (AUTH)					
10	4	Build auth screen html + css (trial throughout)	30/06			
11	5	Build BASE javascripts for all screen	04/07			
12	6	Build the javascripts for auth screen	07/07			
13	7	Trial the functionality of front end	08/07			
14	8	Build the back end + DB (users + passwords+ log in status)	13/07			
15	9	Connect the components and trial + test	13/07			
16	▶ COMPONENT DEVELOPMENT (ONBOARDING)					
17	10	Build onb screen html + css (trial throughout)	15/07			
18	11	Build the javascripts for onb screen (and add some more in the base js files)	17/07			
19	12	Trial the functionality of front end	18/07			
20		Build the back end + add subjects/topics files	21/07			
21	13	Connect the components and trial + test	21/07			
22	▶ COMPONENT DEVELOPMENT (DASHBOARD)					
23	14	Build dashboard screen html + css (trial throughout)	23/07			
24	15	Build the javascripts for dsb screen (and add some more in the base js files)	25/07			
25		Trial the functionality of front end	25/07			
26		Build the back end + DB (add log review + add subject/topic functions), progress graph)	29/07			
27		Build the scheduling algorithm (back end AND js for visual update)	02/08			
28	16	Connect the components and trial + test	02/08			
29	▶ COMPONENT DEVELOPMENT (SUBJECT DETAIL)					
30	17	Build subject detail html + css (trial throughout)	04/08			
31	18	Build the javascripts for subject detail screen (and add some more in the base js files)	06/08			
32	19	Trial the functionality of front end	07/08			
33	20	Build the back end + DB (review count, show review evidences from db)	09/08			
34	21	Connect the components and trial + test	10/08			
35	22					



Task 3: Develop and Trial Components (A-M)

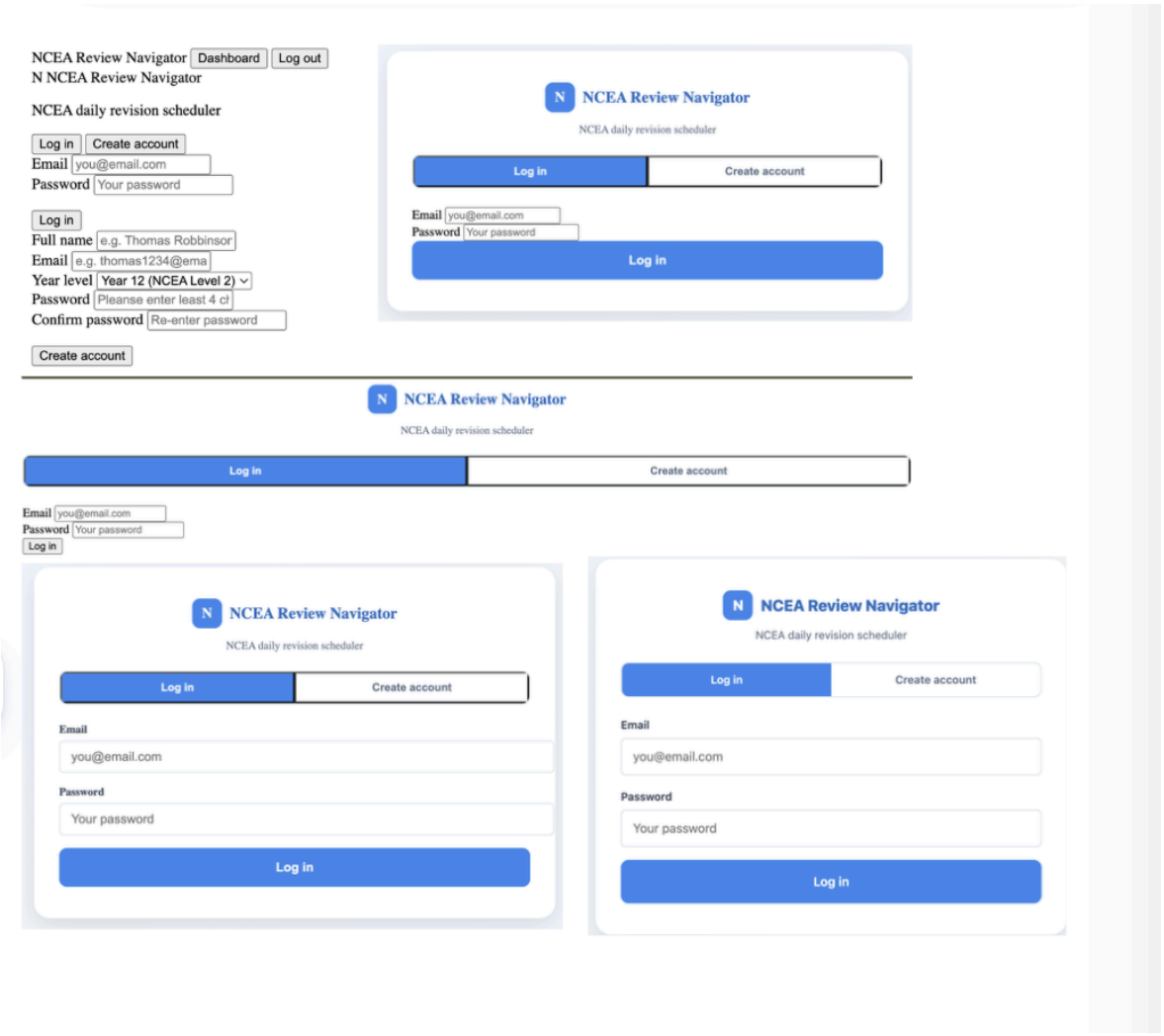
Development stage

Component 1:Log in/ account creation form
<u>Screenshot of the high fidelity</u>



Any evidence of development you wish to include

Here are some links that I used as reference and guide to build this:
E.S Modules that most of my JS based on: JavaScript modules
And event delegation (for app.js): Event delegation + DOM manipulation (for the visual interactivity of the web): How to manipulate the DOM in Vanilla JavaScript.
These links are used for establishing the js BASE (DOM, router, config, main, icon, state, debug), but I do gradually add more features to these bases as it progresses. used for all screens + the whole web needed to function. I will build that first, then the auth screen.



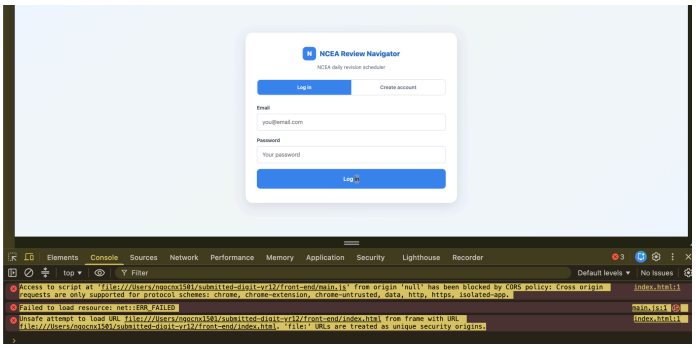
Trialling

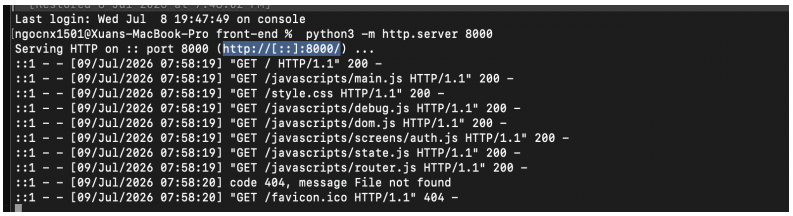
1. **HTML+ CSS (Developing + trialing from 24th of june to 27th of june)**

1. HTML + CSS (24th–27th June):
My first attempt showed all the info and the blue buttons working, but the card container wasn't displaying properly and the whole thing needed a cleaner template. Once I separated the background from the card (white card, grey background) and added a proper title, it looked clean but I'd lost the tab styling on the sign in/create account buttons along the way. I fixed the tab styling, and softened the border thickness and corner radius, and the page ended up looking exactly how I wanted.

2. **Javascript (Developing + Trialing from 28th of June to 9th July)**

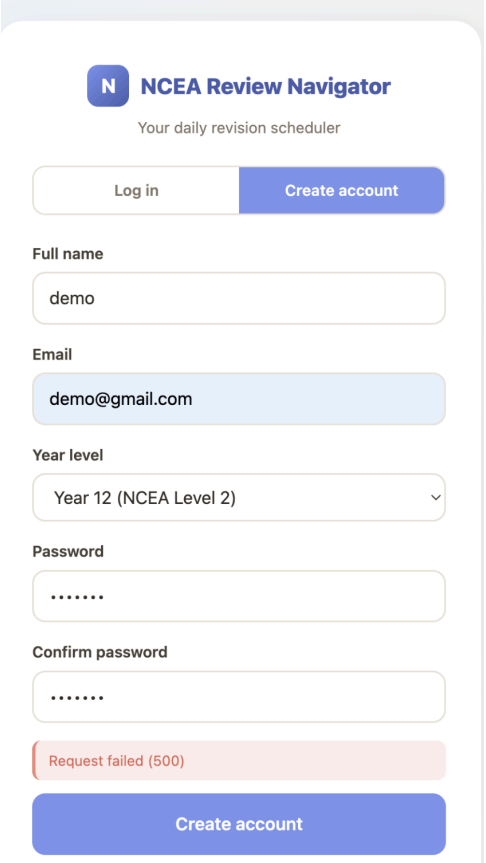
1.First trialling:
First trialling (CORS) I expected to double-click index.html and have my JS modules run straight away in Chrome. Instead the console is filled with CORS errors, because opening a file through file:// blocks ES Module imports as a browser security restriction. I fixed this by running a local server instead (python3 -m http.server 8000) and loading the site through http://localhost:8000, which dropped the restriction completely.



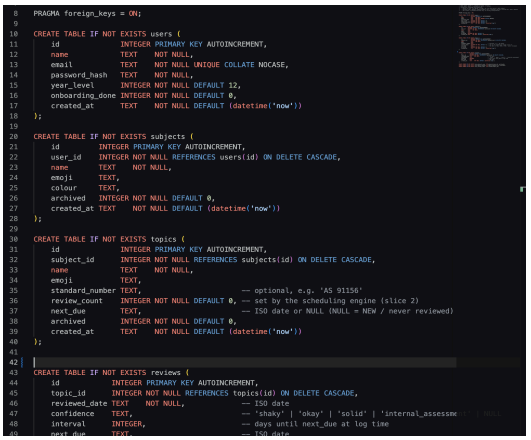


2. Backend + DB (Trialing from 9th of July to 14th July)
Signing up a test account gave a 500 error:
I was trialing to see whether the backend worked. And the console log record sqlite3.OperationalError: no such table: users. My db.py had created the navigator.db file, but schema.sql was still blank, so there was nothing to insert into.I could have just typed CREATE TABLE commands directly into the sqlite3 terminal to patch the immediate problem.So my backend cant operate until db created/ I decided to write a proper schema.sql file instead, because that gives me one single blueprint I can rerun any time the database needs rebuilding like on a different computer, or after wiping test data, rather than having to remember and retype the same commands by hand every time. I wrote the actual CREATE TABLE statements (users, subjects, topics, reviews, linked with foreign keys). I search to write the SQL commands through :
SQLite Foreign Key Support to learn foreign keys and indexes, and SQlite tutorial from SQLite Tutorial +
Python SQLite Tutorial: Build a Python project with a SQLite database and also relational database design from
A Beginner's Guide to Designing a Relational Database (Databases 101) .
After following database tutorials, I realised storing everything in a single flat

table (my init approach) would cause massive data redundancy, as user info would have to be repeatedly stored alongside every single topic and review logged. Instead, a better approach I found through the tutorial is splitting the data into four relational tables (users, subjects, topics, reviews) linked by foreign keys that normalizes the database so each piece of information is stored cleanly in one place. I chose the four-table approach because it keeps the application efficient and scalable as students add more study items over time. This relational structure allows SQLite to handle automatic cascading deletes, instantly wiping all associated subjects, topics, and reviews whenever a user account is removed. Then rerun db.py to build the tables, and signup went through with a 201 created response.




```
back-end % python db.py
Initialised database at 'navigator.db'
(venv) ngocnx1501@Xuans-MacBook-Pro back-end % flask --app app run --port 5050
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on http://127.0.0.1:5050
Press CTRL+C to quit
127.0.0.1 - - [14/Jul/2026 10:09:51] "OPTIONS /api/auth/signup HTTP/1.1" 200 -
[trace] POST /api/auth/signup email='demo@gmail.com'
[trace] signup -> 201 created + logged in
```



```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS users (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT    NOT NULL,
    email           TEXT    NOT NULL UNIQUE COLLATE NOCASE,
    password_hash   TEXT    NOT NULL,
    year_level      INTEGER NOT NULL DEFAULT 12,
    onboarding_done INTEGER NOT NULL DEFAULT 0,
    created_at      TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS subjects (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name        TEXT    NOT NULL,
    emoji       TEXT,
    colour      TEXT,
    archived    INTEGER NOT NULL DEFAULT 0,
    created_at  TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS topics (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_id  INTEGER NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
    name        TEXT    NOT NULL,
    emoji       TEXT,
    standard_number TEXT,                   -- optional, e.g. 'AS 91150'
    review_count    INTEGER NOT NULL DEFAULT 0, -- set by the scheduling engine (slice 2)
    next_due        TEXT,                   -- ISO date or NULL (NULL = NEW / never reviewed)
    archived        INTEGER NOT NULL DEFAULT 0,
    created_at      TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS reviews (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    topic_id    INTEGER NOT NULL REFERENCES topics(id) ON DELETE CASCADE,
    reviewed_date TEXT    NOT NULL,         -- ISO date
    confidence  TEXT,                       -- 'shaky' | 'okay' | 'solid' | 'internal_assessme
    interval    INTEGER,                    -- days until next_due at log time
    next_due    TEXT                        -- ISO date
```

Testing / user feedback

What did the user do?	What do you expect to happen	What actually happens + feedback	What do you need to change

The test user opened the signup tab to create a new profile. They filled out their name and email, but they typed a short 3-character password ("123") into the password field and clicked "Create account". (testing HTML + CSS + JS)	The system should reject the submission, which keeps the user on the signup page, and clearly guide them on how to fix their password strength.	The code successfully caught the validation error using the "MIN_PASSWORD_LEN" and displayed a text block at the bottom of the form saying: "Your password must be at least 6 characters." Feedback: "The system didn't break, which is good, but the error text at the very bottom of the box was easy to miss because my eyes were looking down at the button I just clicked. I didn't realise why it failed at first. I wish the actual password box/texts turned red or shook so I knew exactly where the mistake was immediately.". And I also find that the color is quite industrial-like? Sort of too bright	I need to expand the doSignup function in auth.js. Instead of only changing the text content of the global #signup-err box, I need to add a line that applies a CSS error class (like .input-error { border: 2px solid red; }) directly to the #signup-pw element to give the user a clear visual of the error.

		and mechanics, i would like a calmer, duller color would be nicer.	
Testing backend I set up a user feedback testing where I asked a classmate to test my login system (backend now too). I told them to type in an email that I knew wasn't in the database. Next, I had them type a real email but with a completely wrong password. (Trialing backend)	My classmate expected the app to give them specific feedback, like "That email doesn't exist" or "Password incorrect", so they would know exactly what mistake they made.	In both test cases, my backend sent back the exact same error message: "Incorrect email or password." so my stakeholder gave me feedback saying: "The error message is annoying and confusing because it doesn't tell me which box I got wrong."	Why does this happen? This happened because of the security architecture I wrote in my routes/auth.py file. I intentionally combined the email check and password check into one if statement: if the row is None or not verify_password(...). I had to explain to my user tester that if the backend explicitly tells you "That email doesn't exist," a hacker could use a script to test millions of random emails on my site until it says "Password incorrect." That would let hackers harvest a list of real student emails using my app.

			Even though the user feedback seems valid to me, I decided not to change the backend security code because protecting user accounts from hackers is much more important, especially to maintain safety and security for my users. And as a result, the backend successfully handles bad login attempts securely, which blocks hackers from guessing valid emails.
I had another classmate sign up for a new account. During signup, their mobile phone auto-capitalised the first letter of their email, so they registered as Chloe@gmail.com. A few minutes later, I asked them to log out and try logging back in	I expected the backend to recognise it as the exact same person and log them in smoothly, because email addresses on the internet are usually not case-sensitive.	The backend completely blocked them and threw a 401 Unauthorised error. My user tester was very confused and said, "It's telling me my password or email is wrong, but this is my exact email!"	When I looked into my SQLite database setup, I can see that SQLite text searches are strictly case-sensitive by default. To the database, a capital J and a lowercase j look like two completely different things. My code was

using all lowercase letters (chloe@gmail.com).			running SELECT * FROM users WHERE email = ?, which means it was looking for a lower case account that didn't exist.

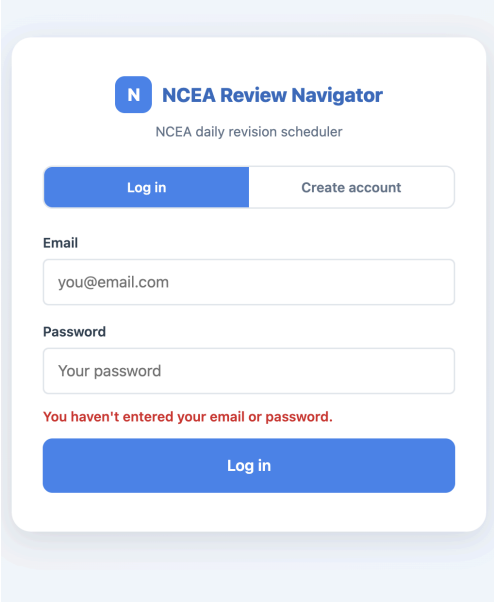
Trialling the changes

I do think the first user feedback is accurate because highlighting the whole input box red would make the error more visible to users, especially since many users may not use computers frequently or have cognitive issues and may find it more difficult to identify the error. So I add

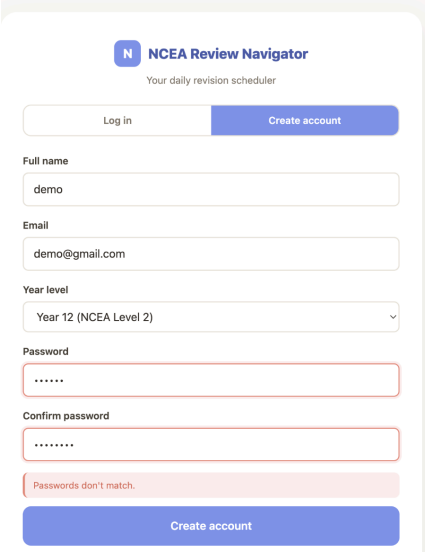
```
.input-error, .input-error:focus {
  border-color: var(--red-500);
  box-shadow: 0 0 0 3px var(--red-50);
}
```

And I take into consideration the users' testing, and I also ask a lot of my friends (stakeholders) and they all prefer lighter colours so I also adjust the whole color palette of the current web into a calmer one.
And as I trial (using python3 -m http.server 8000):
The outcome came out exactly as I expected.

Before (no box highlighted) + different color palette:



After:



——————————

So I go into my backend routes/auth.py file and adjust the SQL login query. I added a special SQLite command called COLLATE NOCASE right at the end of the line:

This will tell the database to completely ignore capital letters whenever it is double-checking emails during a login request.
Then I re-ran the test with my classmate. They were able to log in by typing chloe@gmail.com, Chloe@gmail.com, or even CHLOE@GMAIL.COM. The backend handled it nicely every single time, which made the app way more reliable for real users.

```
    db = get_db()
    row = db.execute("SELECT * FROM users WHERE email = ? COLLATE NOCASE", (email,)).fetchone()
    if row is None or not verify_password(row["password_hash"], password):
        print("[trace] login -> 401 bad credentials", flush=True)  # debug
        raise ApiError("Incorrect email or password.", status=401)

    print("[trace] login -> 200 OK", flush=True)  # debug
    return jsonify(token=encode_token(row["id"]), user=user_public(row))
```

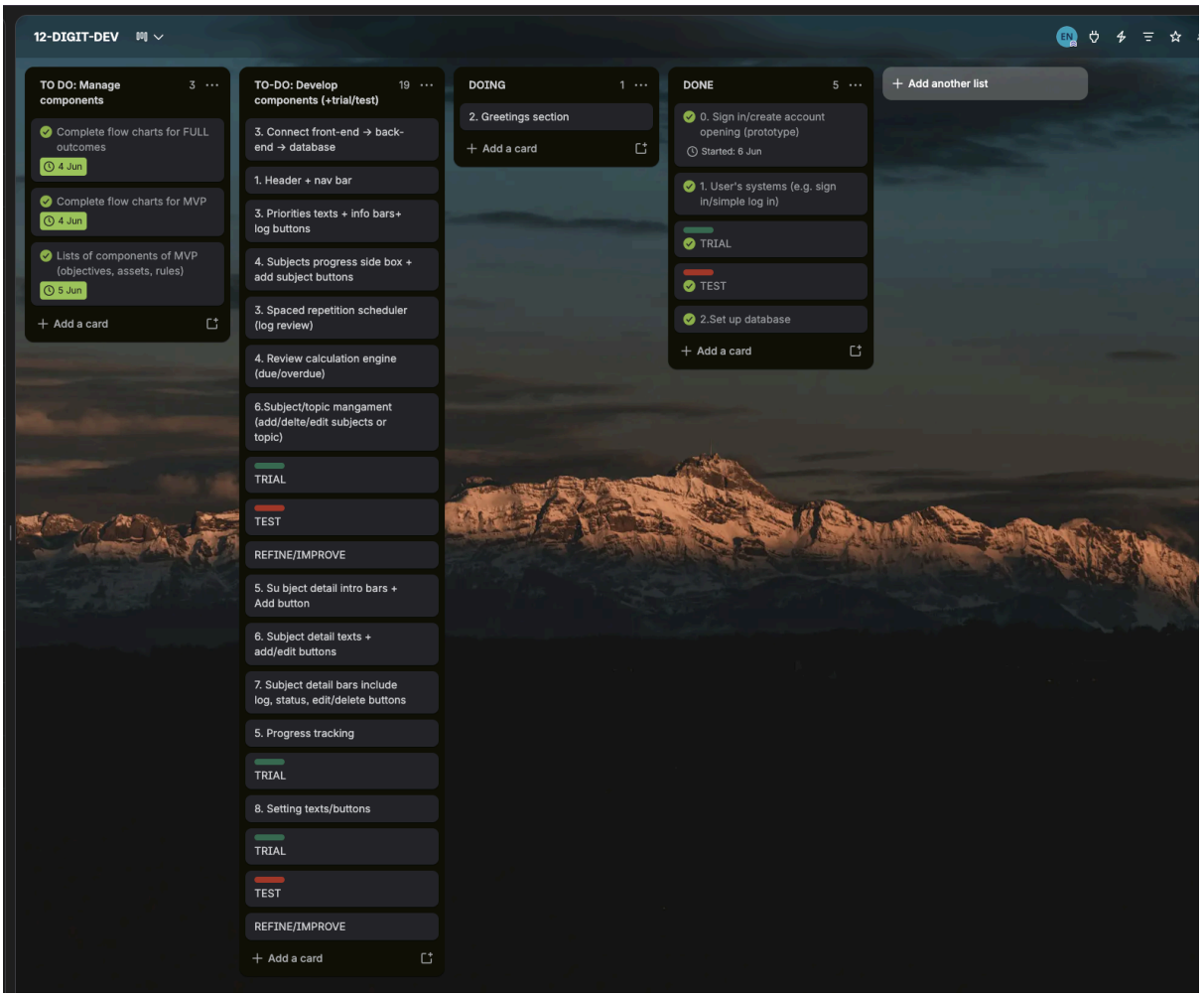
My updated project management :

Development PROJECT MILESTONE TRACKER

STATUS KEY: Completed | Revised/Late | Rescheduled | Upcoming

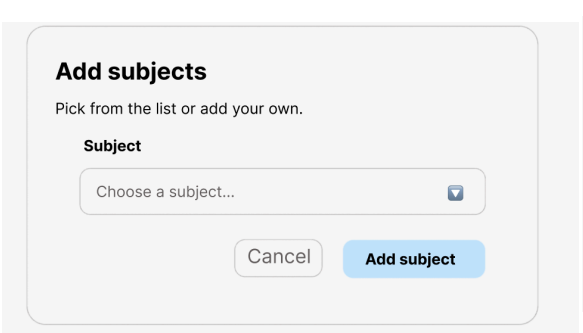
#	Milestone / Task	Deadline	Days took to complete	Time completed	Status
▶	**SETUP**				
1	Set up git + project folder	22/06/2026	1	21/06	Completed
2	Download python + technology needed	24/06	1	27/06	Completed/Late
3	Set up basic html + css+ js files	25/06	1	26/06	Completed/Late
▶	**COMPONENT DEVELOPMENT (AUTH)**				
4	Build auth screen html + css (trial throughout)	30/06	2	28/06	Completed
5	Build BASE javascripts for all screen	04/07	3	01/07	Completed
6	Build the javascripts for auth screen	07/07	2	03/07	Completed
7	Trial the functionality of front end	08/07	1	03/07	Completed
8	Build the back end + DB (users + passwords+ log in status)	13/07	4	06/07	Completed
9	Connect the components and trial + test	13/07	1	06/07	Completed
▶	**COMPONENT DEVELOPMENT (ONBOARDING)**				
10	Build onb screen html + css (trial throughout)	15/07			Upcoming
11	Build the javascripts for onb screen (and add some more in the base js files)	17/07			Upcoming
12	Trial the functionality of front end	18/07			Upcoming
	Build the back end + add subjects/topics files	21/07			Upcoming
13	Connect the components and trial + test	21/07			Upcoming
▶	**COMPONENT DEVELOPMENT (DASHBOARD)**				
14	Build dashboard screen html + css (trial throughout)	23/07			
15	Build the javascripts for dsb screen (and add some more in the base js files)	25/07			
	Trial the functionality of front end	25/07			
	Build the back end + DB (add log review + add subject/topic functions), progress graph)	29/07			
	Build the scheduling algorithm (back end AND js for visual update)	02/08			
16	Connect the components and trial + test	02/08			
▶	**COMPONENT DEVELOPMENT (SUBJECT DETAIL)**				
17	Build subject detail html + css (trial throughout)	04/08			
18	Build the javascripts for subject detail screen (and add some more in the base js files)	06/08			
19	Trial the functionality of front end	07/08			
20	Build the back end + DB (review count, show review evidences from db)	09/08			
21	Connect the components and trial + test	10			
22					

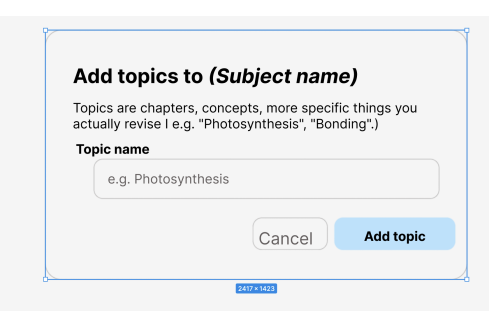
My updated trello:

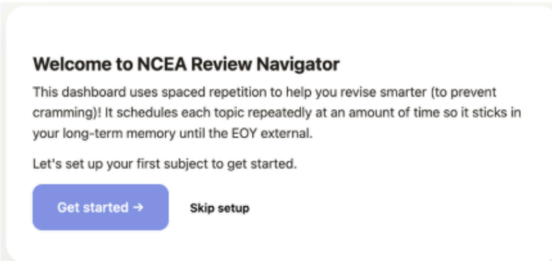


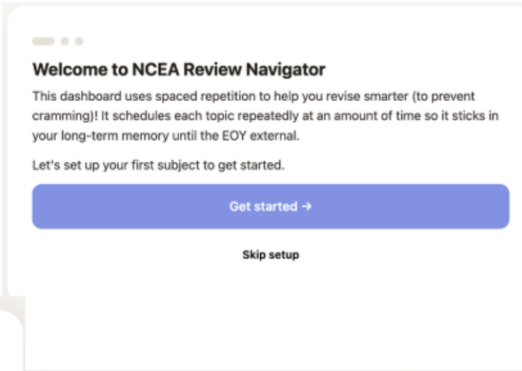
Component 2:Onboarding

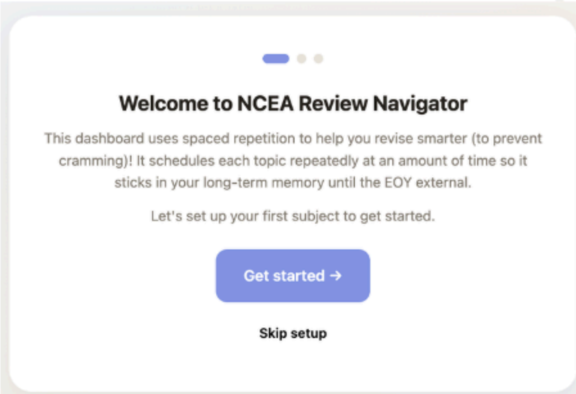
Screenshot of the high fidelity

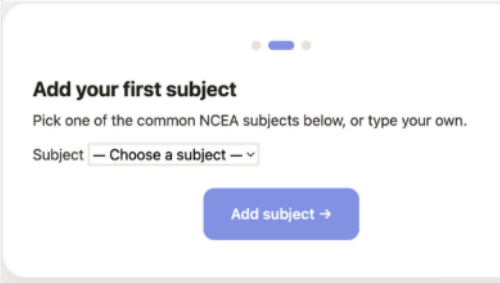


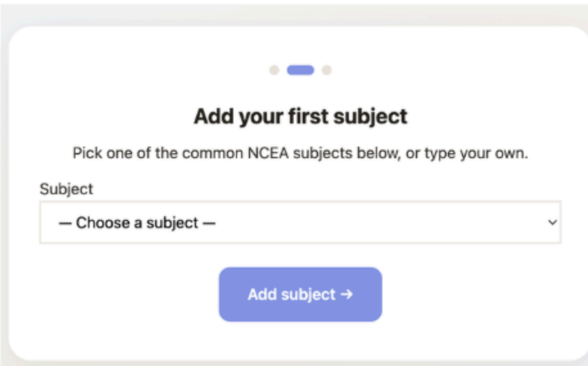



Welcome to NCEA Review Navigator
This dashboard uses spaced repetition to help you revise smarter (to prevent cramming)! It schedules each topic repeatedly at an amount of time so it sticks in your long-term memory until the EOY external.
Let's set up your first subject to get started.
Get started →
Skip setup


Welcome to NCEA Review Navigator
This dashboard uses spaced repetition to help you revise smarter (to prevent cramming)! It schedules each topic repeatedly at an amount of time so it sticks in your long-term memory until the EOY external.
Let's set up your first subject to get started.
Get started →
Skip setup


Welcome to NCEA Review Navigator
This dashboard uses spaced repetition to help you revise smarter (to prevent cramming)! It schedules each topic repeatedly at an amount of time so it sticks in your long-term memory until the EOY external.
Let's set up your first subject to get started.
Get started →
Skip setup


Add your first subject
Pick one of the common NCEA subjects below, or type your own.
Subject
— Choose a subject —
Add subject →


Add your first subject
Pick one of the common NCEA subjects below, or type your own.
Subject
— Choose a subject —
Add subject →

Trialling

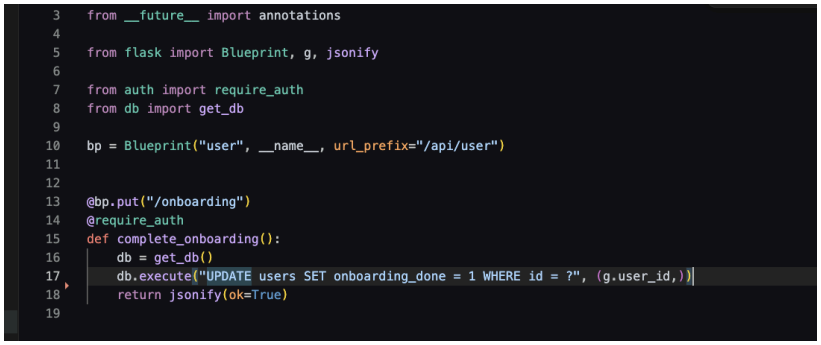
I register a new acc on my web. Because new accounts have onboarding_done set to 0 in the database, the frontend automatically redirected me to the onboarding screen. I selected "Year 12" as my year level, chose my study preferences, and clicked the "Finish Setup" button.

What I expected? I expected the frontend to send a POST request to my /api/auth/onboard route. The backend should update my user row in SQLite to change onboarding_done to 1 and year_level to 12, and then log me into the main dashboard.

What happened? The app told me "Setup Complete!" and let me into the dashboard. However, when I refreshed the browser page, it threw me right back onto the onboarding screen as if I had never done it!

Why did it happened? I opened my backend terminal and didn't see any error crashes, which was confusing. I decided to open my navigator.db file using SQLite Viewer and looked at the users table. My user row still had onboarding_done set to 0 and year_level as NULL.

I looked at my Python code in routes/user.py and found the bug:



```python
from __future__ import annotations

from flask import Blueprint, g, jsonify

from auth import require_auth
from db import get_db

bp = Blueprint("user", __name__, url_prefix="/api/user")


@bp.put("/onboarding")
@require_auth
def complete_onboarding():
    db = get_db()
    db.execute("UPDATE users SET onboarding_done = 1 WHERE id = ?", (g.user_id,))
    return jsonify(ok=True)

```

Because I forgot to write db.commit(), SQLite treated my change as a temporary draft. The second the network request ended, Python discarded the change instead of permanently writing it to the database file.

What did I do next? I added db.commit() immediately after the update query in my Python route, stopped the server, and restarted it.

```python
from __future__ import annotations

from flask import Blueprint, g, jsonify

from auth import require_auth
from db import get_db

bp = Blueprint("user", __name__, url_prefix="/api/user")


@bp.put("/onboarding")
@require_auth
def complete_onboarding():
    db = get_db()
    db.execute("UPDATE users SET onboarding_done = 1 WHERE id = ?", (g.user_id,))
    db.commit()
    return jsonify(ok=True)

```

The result= I deleted my test user from the db, signed up fresh, completed the onboarding wizard again, and hit refresh. This time, the screen stayed on the dashboard. When I checked SQLite, the user row successfully showed onboarding_done = 1.

Testing / user feedback

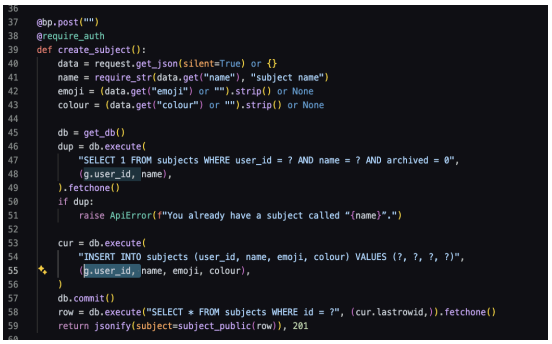
What did the user do?	What do you expect to happen	What actually happens	What do you need to change
I did a usability and security test with a classmate for the onboarding screen. I had them sign up as a new user ("User 2") on my computer right after I had finished setting up my own test account ("demo"), where I had added two subjects: "Digital Technologies" and "Physics".	I expect my web to be able to sign my classmate acc in and preview the dashboard page showing the subjecst.	My user tester told me that they see my subjects sitting on their dashboard. They immediately pointed out: "Why am I seeing your school subjects? Is this web sharing my private study data with other students?" It was a massive privacy and database security fail that my web fails to cover.	I go into my backend Python code for the subject route and check for the bug.

Development for changes

```python

@bp.post("")
@require_auth
def create_subject():
    data = request.get_json(silent=True) or {}
    name = require_str(data.get("name"), "subject name")
    emoji = (data.get("emoji") or "").strip() or None
    colour = (data.get("colour") or "").strip() or None

    db = get_db()
    dup = db.execute(
        "SELECT 1 FROM subjects WHERE user_id = ? AND name = ? AND archived = 0",
        (g.user_id, name),
    ).fetchone()
    if dup:
        raise ApiError(f"You already have a subject called '{name}'.")

```

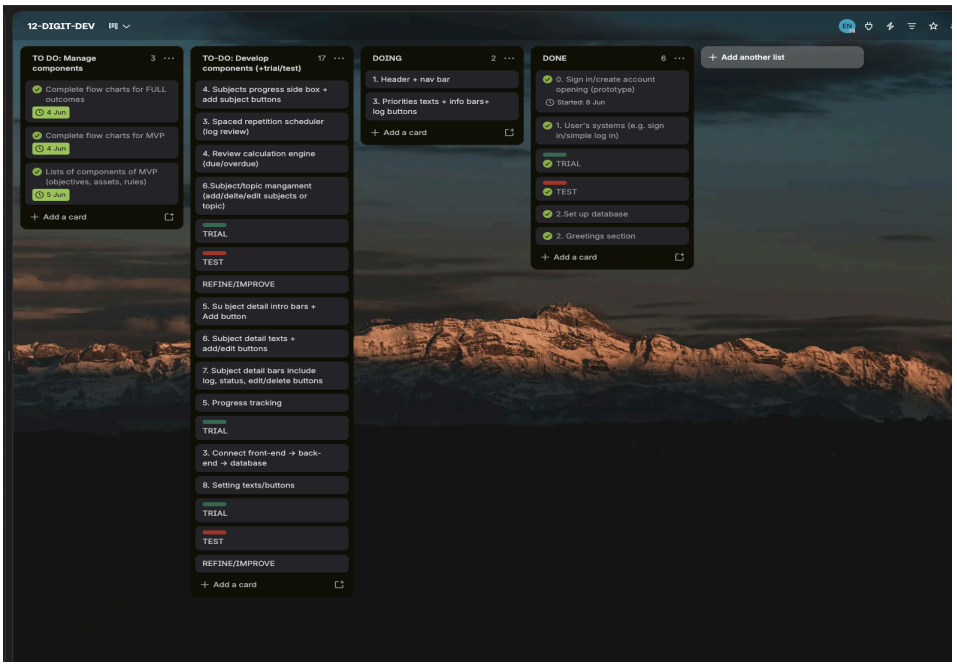
When the frontend requested the list of subjects, my SQL query was:
I had completely forgotten to

filter the database query.SQLite was dumping every single subject in the entire table to whoever asked for it, totally ignoring who was actually logged in.



```python
@bp.post("")
@require_auth
def create_subject():
    data = request.get_json(silent=True) or {}
    name = require_str(data.get("name"), "subject name")
    emoji = (data.get("emoji") or "").strip() or None
    colour = (data.get("colour") or "").strip() or None

    db = get_db()
    dup = db.execute(
        "SELECT 1 FROM subjects WHERE user_id = ? AND name = ? AND archived = 0",
        (g.user_id, name),
    ).fetchone()
    if dup:
        raise ApiError(f"You already have a subject called “{name}”.")

    cur = db.execute(
        "INSERT INTO subjects (user_id, name, emoji, colour) VALUES (?, ?, ?, ?)",
        (g.user_id, name, emoji, colour),
    )
    db.commit()
    row = db.execute("SELECT * FROM subjects WHERE id = ?", (cur.lastrowid,)).fetchone()
    return jsonify(subject=subject_public(row)), 201
```

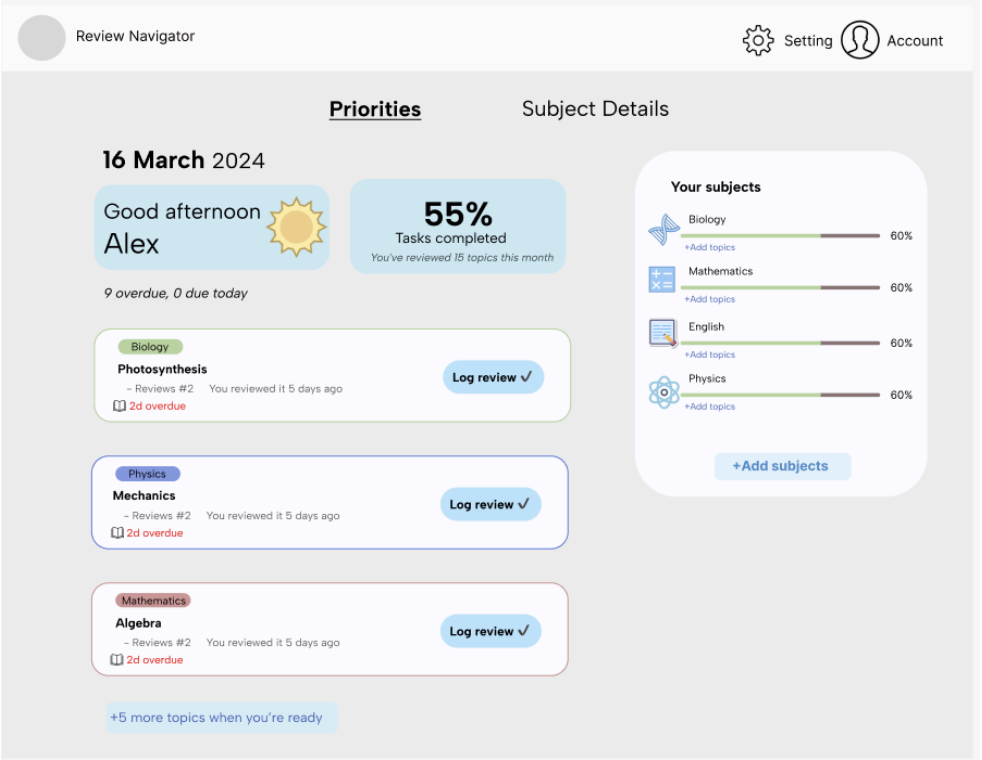
I adjusted the Python route to use my @require_auth bouncer. Because the bouncer safely extracts the logged-in user's ID from their secure JWT token and saves it in g.user_id, I updated the SQL query to look for subjects belonging to that specific user:
We (i ask the stakeholder) ran the user test again. When User 2 logged in, their dashboard was completely empty. They successfully added "Chemistry" (which was saved in SQLite under their user ID). When we switched back to User 1, my dashboard only showed "Physics" and "Digital Technologies". My user tester confirmed that the data leakage was fixed and their account feels secure.



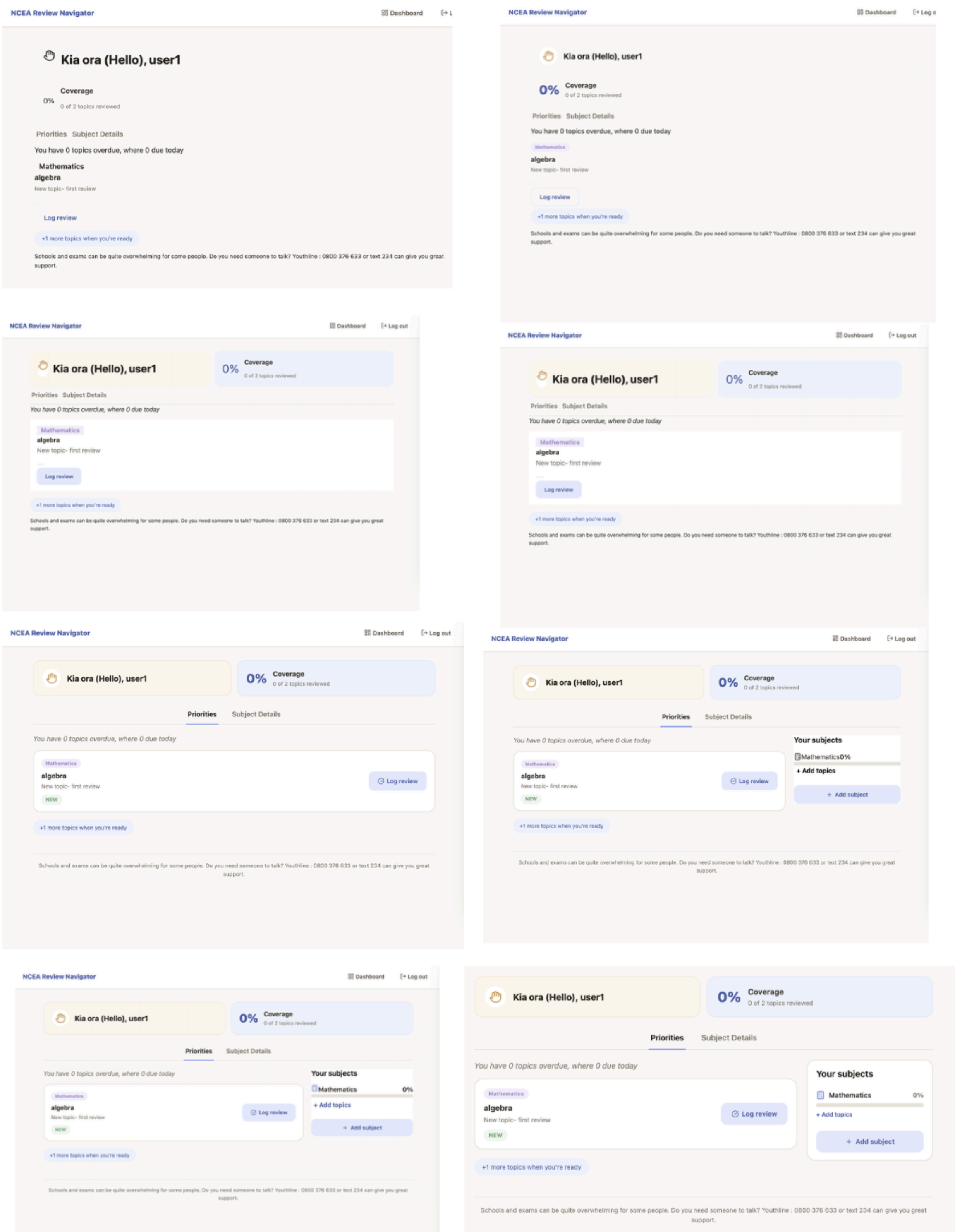
	A	B	C	D	E	F
1	**Development PROJECT MILESTONE TRACKER**					
2						
3		STATUS KEY: Completed Revised/Late Rescheduled Upcoming				
4	**#**	**Milestone / Task**	**Deadline**	**Time took to complete**	**Time completed**	**Status**
5	▶	**SETUP**				
6	1	Set up git + project folder	22/06/2026	1	21/06	Completed
7	2	Download python + technology needed	24/06	1	27/06	Completed/Late
8	3	Set up basic html + css+ js files	25/06	1	26/06	Completed/Late
9	▶	**COMPONENT DEVELOPMENT (AUTH)**				
10	4	Build auth screen html + css (trial throughout)	30/06	2	28/06	Completed
11	5	Build BASE javascrippts for all screen	04/07	3	01/07	Completed
12	6	Build the javascripts for auth screen	07/07	2	03/07	Completed
13	7	Trial the functionality of front end	08/07	1	03/07	Completed
14	8	Build the back end + DB (users + passwords+ log in status)	13/07	4	06/07	Completed
15	9	Connect the components and trial + test	13/07	1	06/07	Completed
16	▶	**COMPONENT DEVELOPMENT (ONBOARDING)**				
17	10	Build onb screen html + css (trial throughout)	15/07	1	07/07	Completed
18	11	Build the javascripts for onb screen (and add some more in the base js files)	17/07	1	08/07	Completed
19	12	Trial the functionality of front end	18/07	1	08/07	Completed
20		Build the back end + add subjects/topics files	21/07	4	11/07	Completed
21	13	Connect the components and trial + test	21/07	1	12/07	Completed
22	▶	**COMPONENT DEVELOPMENT (DASHBOARD)**				
23	14	Build dashboard screen html + css (trial throughout)	23/07			Upcoming
24	15	Build the javascripts for dsb screen (and add some more in the base js files)	25/07			Upcoming
25		Trial the functionality of front end	25/07			Upcoming
26		Build the back end + DB (add log review + add subject/topic functions), progress graph)	29/07			Upcoming
27		Build the scheduling algorithm (back end AND js for visual update)	02/08			Upcoming
28	16	Connect the components and trial + test	02/08			Upcoming
29	▶	**COMPONENT DEVELOPMENT (SUBJECT DETAIL)**				
30	17	Build subject detail html + css (trial throughout)	04/08			
31	18	Build the javascripts for subject detail screen (and add some more in the base js files)	06/08			
32	19	Trial the functionality of front end	07/08			
33	20	Build the back end + DB (review count, show review evidences from db)	09/08			
34	21	Connect the components and trial + test	10			
35	22					

Component 3:Dashboard

Screenshot of the high fidelity



Any evidence of development you wish to include
I filled in the html of the dashboard with js because it is supposed to be customised based on users, therefore, the text will be placed by js (take the data from back end).



Trialling

Trial 1:
What did I do to trial my component?
To test the daily review cap on the dashboard, I put in 2 topics of same subject directly into the db using the browser console. Since my topic cap is set to 1 topic per subject, I wanted to force the "+1 more topics" pill to appear and check whether clicking it would actually function and load the rest of the schedule.

What happened?
The dashboard successfully loaded and showed the "+1 more" pill, but when I clicked it, absolutely nothing happened.
+1.. bug

When I inspected my code, I realised two major bugs. First, the pill was just a <p> tag styled with CSS to look like a button and it had no data-action or JavaScript event listener attached to it. Second, the backend was only calculating and sending the number of extra topics (moreCount) to the student, but it wasn't sending the actual topic data. There was literally nothing for the frontend to open.

Decision:
I had to refactor the logic across both the frontend and backend to fix this. In my backend scheduling engine (scheduling.py) and its JavaScript mirror (schedule.js), I added a "rest list" array. This list now actually holds the data for every topic beyond the daily cap, keeping them sorted in the correct overdue-first ranking. On the frontend, I add a real paragraph <button data-action="show-more-topics">. Clicking this now toggles a showAllTopics state flag in the JavaScript, which re-renders the extra priority cards inline on the dashboard, each with a fully working "Log review" button.

```
118 118
119 119         display = active + first_per_subject
120 120         shown = display[:daily_cap]
    121 +
    122 +     # Everything the student could review today, in the same ranked order.
    123 +     # "rest" is what is left after the daily cap, so the dashboard can offer it.
    124 +     first_ids = {i.get("id") for i in first_per_subject}
    125 +     everything = display + [i for i in news if i.get("id") not in first_ids]
    126 +     rest = everything[len(shown) :]
    127 +
121 128         total_reviewable = len(active) + len(news)  # everything overdue/due/new
122 129         return {
123 130             "shown": shown,
    131 +         "rest": rest,
124 132             "moreCount": total_reviewable - len(shown),
125 133             "overdue": overdue,
126 134             "dueToday": due_today,
```

```
    104 +   const firstIds = new Set(firstPerSubject.map((i) => i.id));
    105 +   const everything = [...display, ...news.filter((i) => !firstIds.has(i.id))];
    106 +   const rest = everything.slice(shown.length);
    107 +
100 108     const totalReviewable = active.length + news.length;
101         -   trace('schedule: buildPriorities', { shown: shown.length, overdue, dueToday });
    109 +   trace('schedule: buildPriorities', { shown: shown.length, rest: rest.length, overdue, dueToday });
102 110     return {
103 111       shown,
    112 +     rest,
104 113       moreCount: totalReviewable - shown.length,
105 114       overdue,
106 115       dueToday,
```

And the result:

+1...bugsolve

Trial 2:

What do i expect:

when i click the subject field should display all available preset choices. Choosing a subject (e.g., "English") should fill the field, but re-opening the menu should still allow the user to easily switch to another subject (e.g., "Chemistry").

What happen

Once I clicked "English", the dropdown menu collapsed and refused to show any other subjects. To pick a different subject, I was forced to manually delete the word "English" character-by-character to reveal the list again.

Drop down list glitches

Why does it happen?

I initially used an <input list="add-subj-list"> bound to a <datalist> in my trialing. I chose a datalist because I assumed it was the best tool to allow users to either pick a suggested subject or type a custom one.

But a <datalist> is an autocomplete suggestion engine, not a true select menu. The browser filters the visible suggestions to match whatever string is currently inside the text box. This means that when the user selected "English", the text

field was filled with "English". The browser immediately filtered the 8 suggested options to only those containing the text "English".

So i refactored js/screens/add.js to use a standard HTML <select> menu paired with a hidden text field for custom inputs, where in:

I refactored this to a standard <select> dropdown with preset options plus an "Other" choice that toggles a hidden text <input> field. This proved to be the better approach because it kept all preset choices fully visible at all times and still gave students the clean flexibility to type in custom subjects.

```
<input class="input" id="add-subj"
       list="add-subj-list"
       placeholder="e.g. Biology">
<datalist id="add-subj-list">
  ...8 options...
</datalist>
```

```
<select id="add-subj-select">
  <option value="">— Choose —</option>
  ...8 options...
  <option value="__custom__">
    Other (type your own)</option>
</select>
<div id="add-custom-field" class="hidden">
  <input id="add-subj">
</div>
```

Before:

After

JS Toggle (openAddSubject): I add an onchange listener so selecting "Other" hides the text box and auto-focuses it, so that switching back to a preset hides it again.

```
// Toggle custom input field dynamically
const sel = $('#add-subj-select');
sel.onchange = () => {
  const isCustom = sel.value === '__custom__';
  $('#add-custom-field').classList.toggle('hidden', !isCustom);
  if (isCustom) $('#add-subj').focus();
};
```

Submit (submitAddSubject): It updates the submission logic to read from whichever input control is currently active.

And for the result, all 10 options remain available at all times (8 subjects + placeholder + "Other") and switching options like English -> Chemistry) works instantly with no text clearing required.

Drop-down list result

Testing / user feedback

What did the user	What do you	What actually	What do you

do?	expect to happen	happens	need to change
The user fills out the log reviews form on the dashboard and clicks the "Submit" button to record their study session.	When the form is submitted and the db updates successfully, the system should provide visual feedback saying that it succeeded.	The frontend JavaScript sends the data to the Flask backend and updates the progress meter, but the form simply clears itself without telling the user "Success." This left the tester guessing if their data was actually saved or if the page just refreshed. She said: "Everything works well but after log review, there is no notification or tick display confirmation that it is successful -> hard to know if it works even though progress did change."	So I need to implement a visual state change in the DOM such as a green tick or a temporary "Log Saved!" toast notification, triggered by the 200 OK response from the Fetch API.
Users were log reviews late at night (New Zealand Time) and they noticed that their reviews were being recorded on the wrong set, or topics due "today" were shown as due "tomorrow".	The system should define "today" based on the student's actual local time pacific/auckland, regardless of where the web is hosted in the world.	The tester asked me a question that really made me question about the gap in the current code that I'm writing: “but what if the date is different due to daylight saving etc? Like if I study	The tester makes me realise that I need to stop relying on the host machine's system clock. And force the server to explicitly calculate "today" using New Zealand timezone

		at night, does it count for today or tomorrow?”	rules, that account for Daylight Saving automatically too.
A user adds a new subject with multiple topics and expects all of them to show immediately on the daily study feed.	The user expects a traditional "to do list" where adding 5 items results in 5 items appearing on the screen.	The dashboard strictly limits the feed to ≤1 new topic per subject, per day (prioritising the oldest first) and hides the rest. During live testing, a bug allows multiple topics to slip through, but I change it to enforce the strict limit. The UI now displayed the 1 topic, alongside a message: "+4 more topics when you're ready." User’s feedback: I just added a subject with 5 new topics, but the dashboard is only showing me 1 to study today. Why can't I just see all of them at once? I think it would be better to see all the topics.	I don't think I would change anything in the core logic. This is an intentional design choice that needs justification, though the UI messaging needed to be clearer so users don't think their data was deleted. Why do I still keep this approach? I actively opposed changing this logic because dumping 5 topics on day one creates the exact cognitive overwhelm my web was designed to eliminate. And also introducing 5 topics on the same day violates the core principles of spaced repetition (the

			1-3-7 sequence relies on distribution). Even though the trade-off means a single-subject user only sees one new topic on their first day, the mathematical sequences will fill the feed over the following week, causing the system to self-correct. I keep the strict rule but updated the frontend DOM to clearly display the "+N more when you're ready" text, which provides transparency and freedom (if the users want to log other topics) without compromising my scheduling algorithms.
Development to address changes			
Development to address changes I updated the .then() block in dashboard.js so a successful save triggers a two-stage confirmation instead of relying on the progress bar alone, including the modal showing a green tick and "Log saved!" for 1.1 seconds before closing, then a green toast confirms it again for 3.6 seconds while the dashboard refreshes behind it.			

```
  trace('log-review: saved OK → showing confirmation');
  const done = pending.onDone;
  const topicName = pending.topicName;
  pending = null;

  showSavedState(topicName); // 1. tick + "Log saved!" in the modal
  await new Promise((r) => setTimeout(r, SUCCESS_HOLD_MS));
  closeModal();
  toast('Review logged- nice work!', 'success'); // 2. green toast as it closes
  if (done) await done(); // 3. refresh the screen behind the modal
} catch (e) {
  trace('log-review: submit FAILED', e.message);
  toast(e.message || 'Could not log review');
}
```

——---

Before, I used Python's default date.today(). But free hosting servers run on Coordinated Universal Time. Because UTC is up to 13 hours behind NZ, a student studying at 9 PM in New Zealand was technically logging a review for "yesterday" according to the server. This timezone desync can badly affect my spaced repetition intervals and user experience.

But the timezone issue that the tester pointed out to me made me change my approach to build a clock.py module using Python's zoneinfo library to calculate "today" specifically in Pacific/Auckland time. This includes daylight saving, set through an environment variable rather than hardcoded. This keeps the principle that the server, not the student's local clock , decides what "today" is, so the date can't be wrong depending on where the web is hosted.

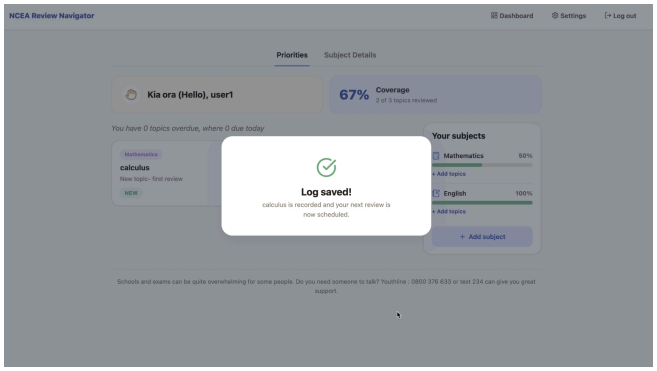
So I watch the overall tutorial online from :
A Deep Dive Into Date And Time In Python to have a deeper understanding of date & time. Instead of just printing that value of time, I need to turn it into a value that my backend engine uses + front end display.

```
mitted-digit-yr12 > back-end > clock.py > ...
  """Server clock
  """

  from __future__ import annotations

  import os
  from datetime import date, datetime
  from zoneinfo import ZoneInfo

  APP_TIMEZONE = os.environ.get("APP_TIMEZONE", "Pacific/Auckland")


  def today() -> date:
      """The current calendar date in the app timezone (the server's 'today')."""
      return datetime.now(ZoneInfo(APP_TIMEZONE)).date()
```

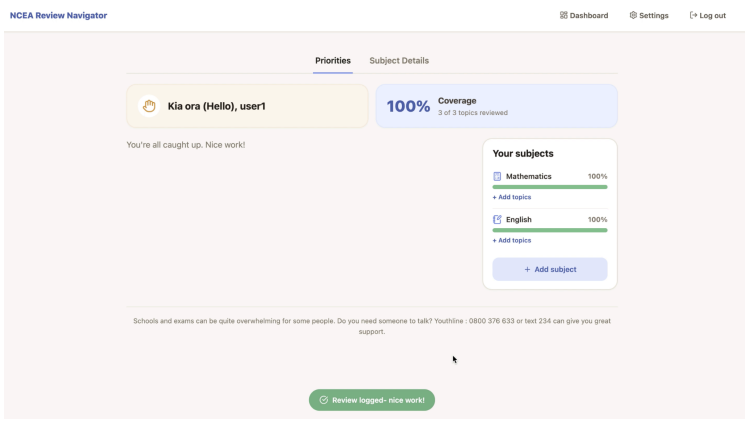
Trialling the changes

I open up my terminal to trial these features and all of them work perfectly.
The green log saved toast:

Stage 1:



Stage 2:



Component 4: Spaced Repetition Scheduler

Trialling

For development, I research online existing open source simple spaced repetition python modules (but their codes are completely different to mine due to different context, but I do use them as a guide then adjust based on my own purpose).
This is the video that guides my code the most because it deals with timedelta & intervals.
Live coding Python.cards: open sourcing simple-spaced-repetition.
Trial 1:
I tested what happens when a student first creates an account and adds a new subject containing 5 new topics, and checks if the daily priority feed displays

them correctly. I expected the space-repetition rules state a student should only get a maximum of 1 new topic per subject per day so they don't get overwhelmed.

But I found a bug that when I added Biology with 5 topics, all 5 topics go into the "Due Today" feed immediately, which pushes out older overdue cards from other subjects.

Why did this happen?
My original algorithm found every topic with a "new" status and pushed it straight into the display list until it hit the daily all topics cap of 5. The code didn't check which subject those topics belonged to, meaning one subject could easily spread the entire daily feed and break the spaced-repetition pacing.

How I developed the changes
I updated the buildPriorities function (in both Python and JavaScript) to use a Set to track which subjects had already shown a new card:

I created an empty seen = new Set() to remember subject IDs.

I sorted all "new" topics by oldest first.
I looped through the new topics. If the subjectId was not in the seen set, I added the topic to the feed and recorded its ID. If the ID was already in the set, the code skipped it.

```
  const firstPerSubject = [];
  const seen = new Set();
  news.forEach((i) => {
    if (!seen.has(i.subjectId)) {
      seen.add(i.subjectId);
      firstPerSubject.push(i);
    }
  });
  firstPerSubject.sort((a, b) => (a.id || 0) - (b.id || 0));

  const display = [...active, ...firstPerSubject];
  const shown = display.slice(0, cap);
  const totalReviewable = active.length + news.length;
  trace('schedule: buildPriorities', { shown: shown.length, overdue, dueToday });
  return {
    shown,
    moreCount: totalReviewable - shown.length,
    overdue,
    dueToday,
  };
}
```

The feed now has a filter . If Biology has 5 new topics, only 1 appears in the feed. The other 4 are mathematically put back for future days, so the web can enforce the "no overwhelm" design choice.
—--
Trial 2:
What did I do?
I click the "Log Review" button on a new topic that had never been studied before.
The Python log_review function should realise this is the first review, set the review_count to 1, and set the next due date to 1 day in the future (based on INTERVALS = {1: 1}).

What happened?
The server crashed and returned a 500 Internal Server Error. The topic was not saved.

Why did this happen? (Root Cause)

Clicking "Log Review" on a topic that had never been studied crashed the server with TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'. A brand-new topic doesn't have a reviewCount in the database yet, so it's sent through as null, and Python can't add None + 1.

One option here was to fix this only on the front end, so it always sends 0 instead of null for a topic that's never been reviewed. I decided not to rely on that by itself, because it would mean the backend was only safe if the front end behaved perfectly, and if I ever changed the front end code later, the same crash could come back. So I decided to fix it on the backend instead, with topic.get("reviewCount") or 0. This meant the scheduling engine defends itself no matter what value it's actually sent, which felt like the safer place to put the fix.
My original code was:

```
def log_review(topic: dict, today: date) -> dict:
    """Move a topic one step forward, return what changed."""
    new_count = (topic.get("reviewCount")) + 1
    interval = interval_for(new_count)
    return {
        "review_count": new_count,
        "interval": interval,
        "reviewed_date": today.isoformat(),
        "next_due": next_due_date(today, new_count).isoformat(),
    }
```

After:

```python
def log_review(topic: dict, today: date) -> dict:
    """Move a topic one step forward, return what changed."""
    new_count = int(topic.get("reviewCount") or 0) + 1
    interval = interval_for(new_count)
    return {
        "review_count": new_count,
        "interval": interval,
        "reviewed_date": today.isoformat(),
        "next_due": next_due_date(today, new_count).isoformat(),
    }
```

And as a result the system is now "type safe". A new topic safely falls back to 0 before adding 1, which successfully starts them in the spaced repetition engine without server crash .

Testing / user feedback

What did the user do?	What do you expect to happen	What actually happens	What do you need to change
I want to simulate longterm app usage. The tester repeatedly clicks "Log Review" on a single topic four times in a row to see how the algorithm would handle older material.	I expect the spacing to follow my 1-3-7 day schedule for the first three reviews. For the 4th review (or any review after that), I expect the algorithm to cap the wait time at a max of 14days so the student never completely loses track of the topic.	On the 1st, 2nd, and 3rd reviews, it worked fine. But on the 4th review, the tester found that the front-end dashboard suddenly displayed `"due in NaN days"` (Not a Number), and when i was checking the db show the `nextDue` date was saved as blank/null.	I need to adjust the interval so that a topic can be reviewed more than 3 times.

Trialling the changes

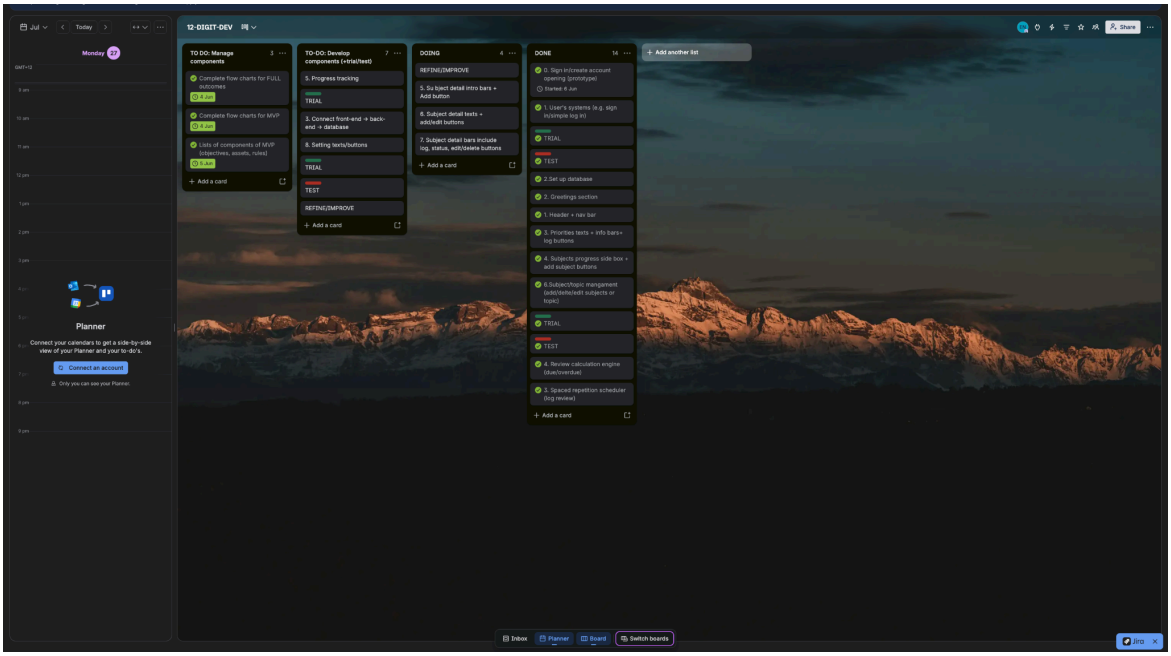
```python
# Review schedule: wait this many days after each review
INTERVALS = {1: 1, 2: 3, 3: 7}  # 1st review = 1 day, 2nd = 3 days, 3rd = 7 days
```

My INTERVALS dictionary only had entries for reviews 1–3, so review 4 returned undefined in JavaScript and a KeyError in Python, which broke the date maths.

I did consider just adding more entries to the INTERVALS dictionary, intervals for review 4, 5, 6 and so on.
But it wouldn't be the best approach because it doesn't actually fix the problem, it just delays it: a student who reviewed a topic 50 times would still hit the exact same crash eventually, just later. So I use .get(review_count, INTERVAL_MAX) instead meant every review count beyond 3 automatically falls back to a 14-day cap, so the fix works no matter how many times a topic gets reviewed, rather than only up to whatever number I happened to type into the dictionary.

```
export const INTERVALS = { 1: 1, 2: 3, 3: 7 };
export const INTERVAL_MAX = 14;

export function intervalFor(reviewCount) {
  return INTERVALS[reviewCount] || INTERVAL_MAX;
}
```

Project management:



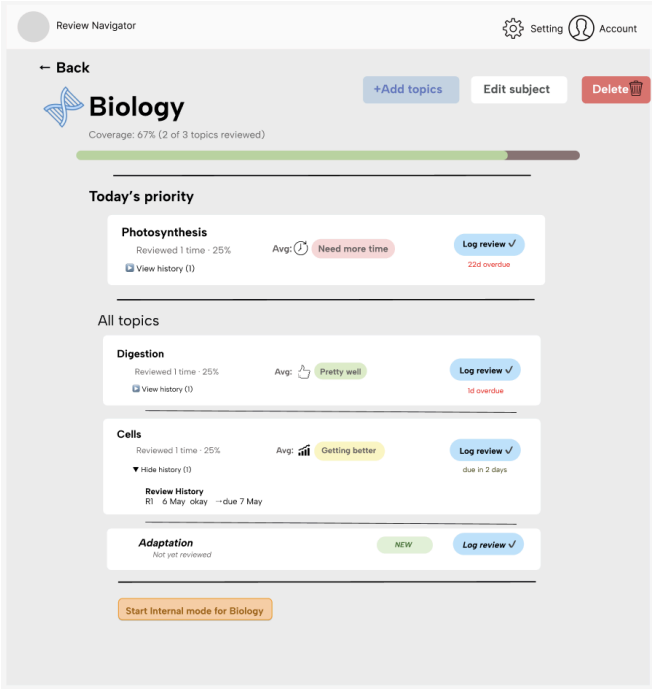
Development PROJECT MILESTONE TRACKER

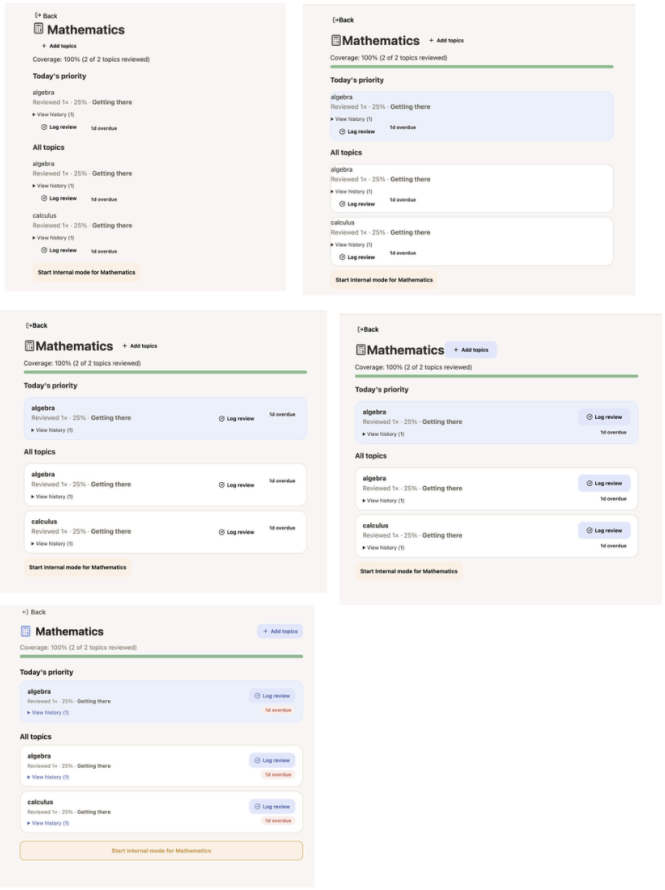
STATUS KEY: Completed | Revised/Late | Rescheduled | Upcoming

#	Milestone / Task	Deadline	Time took to complete	Time completed	Status
▶	**SETUP**				
1	Set up git + project folder	22/06/2026	1	21/06	Completed
2	Download python + technology needed	24/06	1	27/06	Completed/Late
3	Set up basic html + css+ js files	25/06	1	26/06	Completed/Late
▶	**COMPONENT DEVELOPMENT (AUTH)**				
4	Build auth screen html + css (trial throughout)	30/06	2	28/06	Completed
5	Build BASE javascripts for all screen	04/07	3	01/07	Completed
6	Build the javascripts for auth screen	07/07	2	03/07	Completed
7	Trial the functionality of front end	08/07	1	03/07	Completed
8	Build the back end + DB (users + passwords+ log in status)	13/07	4	06/07	Completed
9	Connect the components and trial + test	13/07	1	06/07	Completed
▶	**COMPONENT DEVELOPMENT (ONBOARDING)**				
10	Build onb screen html + css (trial throughout)	15/07	1	07/07	Completed
11	Build the javascripts for onb screen (and add some more in the base js files)	17/07	1	08/07	Completed
12	Trial the functionality of front end	18/07	1	08/07	Completed
	Build the back end + add subjects/topics files	21/07	4	11/07	Completed
13	Connect the components and trial + test	21/07	1	12/07	Completed
▶	**COMPONENT DEVELOPMENT (DASHBOARD)**				
14	Build dashboard screen html + css (trial throughout)	23/07	1	15/07	Completed
15	Build the javascripts for dsb screen (and add some more in the base js files)	25/07	2	16/07	Completed
	Trial the functionality of front end	25/07	1	16/07	Completed
	Build the back end + DB (add log review + add subject/topic functions), progress graph)	29/07	6	23/07	Completed
	Build the scheduling algorithm (back end AND js for visual update)	02/08	3	25/07	Completed
16	Connect the components and trial + test	02/08	1	25/07	Completed
▶	**COMPONENT DEVELOPMENT (SUBJECT DETAIL)**				
17	Build subject detail html + css (trial throughout)	04/08			Upcoming
18	Build the javascripts for subject detail screen (and add some more in the base js files)	06/08			Upcoming
19	Trial the functionality of front end	07/08			Upcoming
20	Build the back end + DB (review count, show review evidences from db)	09/08			Upcoming
21	Connect the components and trial + test	10			Upcoming
22					

Component 5: Subject Detail page

Screenshot of high fidelity





Trialing

1. Front-end (Js):

I opened the subject detail page (e.g., Biology) that had 8 different topics. Some are overdue, some were due today, and some are brand new. I wanted to see if the algorithm successfully picked the most urgent topic to display in the "Today's priority" box at the top of the screen.

What did I expect?
I expect the most urgent topics to appear in the priority box, and the "All topics" list below it to remain in its logical order (the order I added them to the database).

What happened?
 The priority box worked fine, but the "All topics" list underneath got completely scrambled. It lost its original order and matched the priority sorting order (grouping all overdue things at the top).

Why does it happen?

I originally wrote this:

```
const topics = subject.topics || [];
const cov = coverageOf(topics);
const reviewed = topics.filter((t) => (t.reviewCount || 0) >= 1).length;
// Find the most urgent topic to show at the top
const RANK = { overdue: 3, due: 2, new: 1, ok: 0 };
const sorted = topics.sort((a, b) => {
  const ra = statusOf(a);
  const rb = statusOf(b);
  return RANK[rb.status] - RANK[ra.status] || rb.days - ra.days;
});
const priority = sorted[0] && statusOf(sorted[0]).status !== 'ok' ? so
const internal = !!subject.internalMode; // true if internal mode is o
```

I didn't realise that in js, the .sort() function permanently changes the original array. So when my code is sort the list to find the #1 most urgent item for the top of page, it permanently rearranges the master topics array. And when code moves down to render the html for all topic sections (topics.map(...)), it renders the scrambled list.

I researched “js array sort keeping original order” and found out about spread operators (...). The spread operates to unpack all the items in an array and put them into a new temporary array. So the trialing error allows me to see the issue and solve it by adding the spread operators (a different addition) can help my lists of topics be sorted accurately.
So i update my code to create a copy before sorting:

```
const topics = subject.topics || [];
const cov = coverageOf(topics);
const reviewed = topics.filter((t) => (t.reviewCount || 0) >= 1).length;
// Find the most urgent topic to show at the top
const RANK = { overdue: 3, due: 2, new: 1, ok: 0 };
const sorted = [...topics].sort((a, b) => {
  const ra = statusOf(a);
  const rb = statusOf(b);
  return RANK[rb.status] - RANK[ra.status] || rb.days - ra.days;
});
const priority = sorted[0] && statusOf(sorted[0]).status !== 'ok' ? so
const internal = !!subject.internalMode; // true if internal mode is o
```

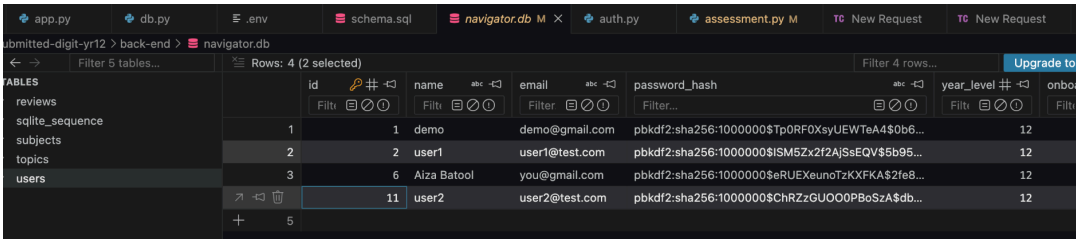
The [...topics] create a clone so the original array is kept.
As a result, the log topic array is kept . Today's priority algorithm gets its own temporary sorted list to find the most urgent but the “all topics”list renders the sequence of arrays as it expected.
—------

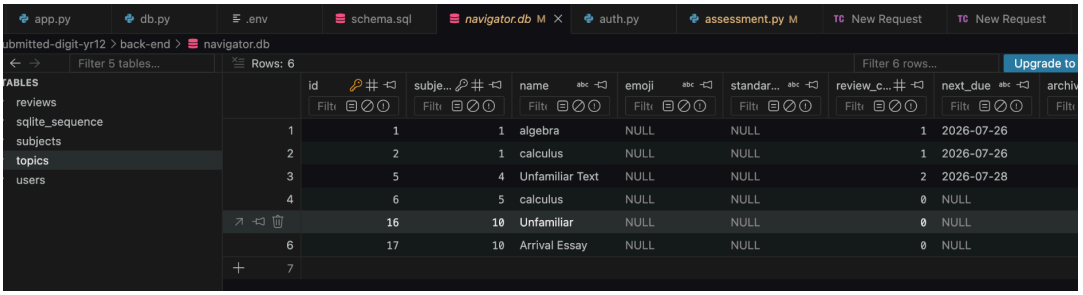
2. I did a security trial on my Flask backend API endpoint (POST /api/assessment-mode-start) to verify whether my web is protected against Broken Object Level Authorization (BOLA).

Even though my frontend UI only shows users their own subjects, an attacker could bypass the user interface entirely using API testing tools like Postman or Thunder Client (this is what i did). If a logged-in user (User 1) sends a raw HTTP request containing the database ID of a subject belonging to another student (User 2), the backend must strictly

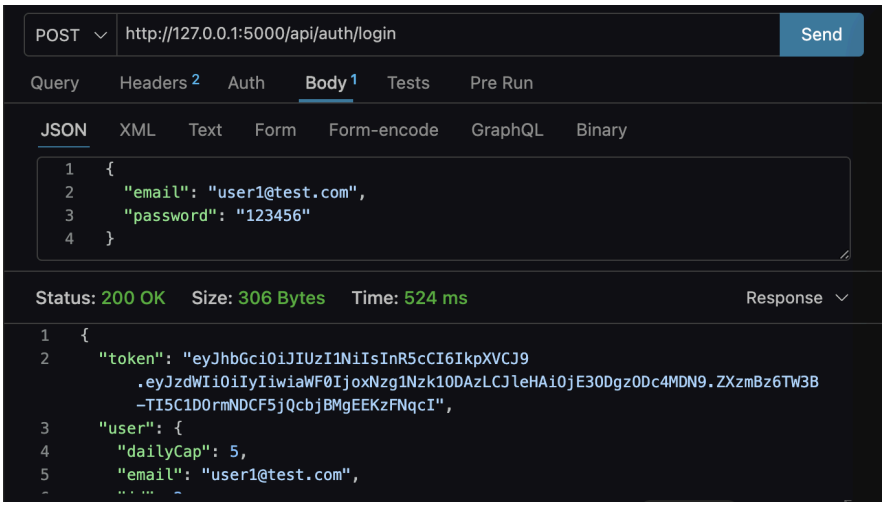
block that request. Without server-side checks, any authenticated user could modify or disrupt another student's data.

How did I do it?
I try to simulate an actual security attack in a controlled environment, I used Thunder Client (a VS Code extension for API testing):
*Note: the created accounts in here are fake/tested, there are no real users used yet so I ddint violate anyone's privacy information.

1. I checked navigator.db to see two different test user accounts existed (user_id = 1 and user_id = 2).
2. 

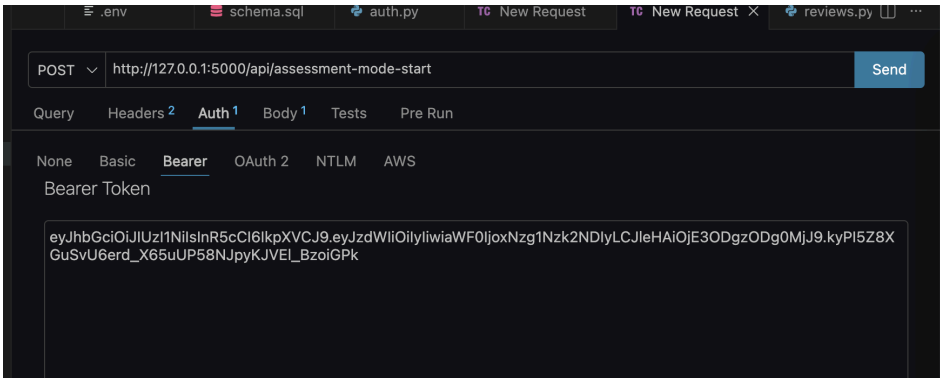
3. I identified topic id = 16, which belongs strictly to User 2.


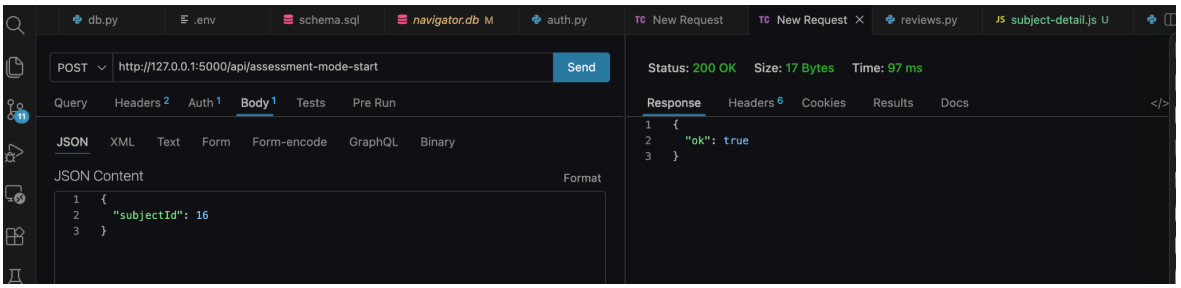
4. I logged in as User 1 through the API to receive a valid JWT Bearer token.



In Thunder Client, I configured a POST request to [http://127.0.0.1:5000/api/assessment-mode-start](http://127.0.0.1:5000/api/assessment

-mode-start) using User 1's JWT token, but passing User 2's subject ID in the JSON body:





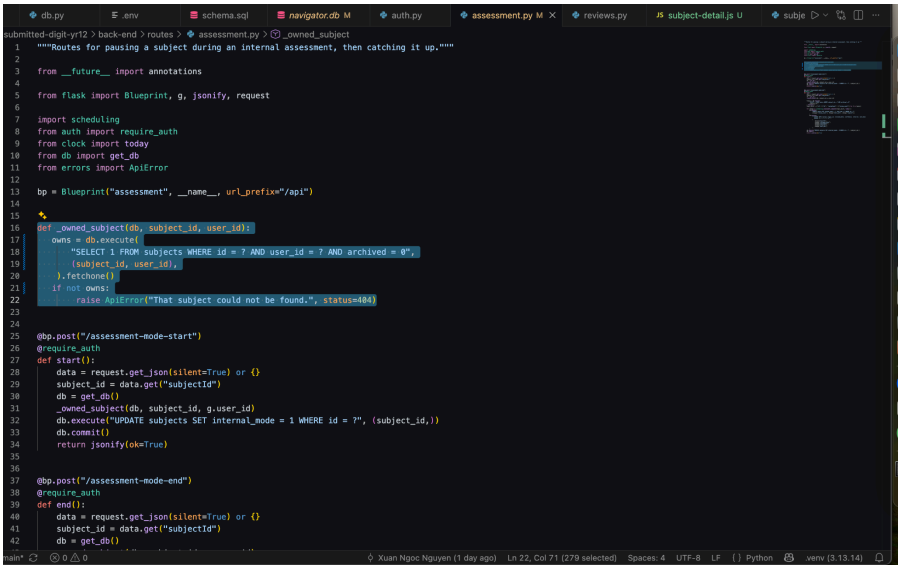
When I clicked Send in Thunder Client: HTTP Status Returned: 200 OK

Response Body: {"ok": true}

This means that the backend ran UPDATE subjects SET internal_mode = 1 WHERE id = 16, changing User 2's subject mode in SQLite without their knowledge.

This proved to me that the @require_auth decorator alone is not enough for safety. @require_auth only verifies who the user is (Authentication), but it does not check what resources they are allowed to touch (Authorisation). The API was very vulnerable to attackers.

I implemented and activated the _owned_subject helper function in routes/assessment.py:



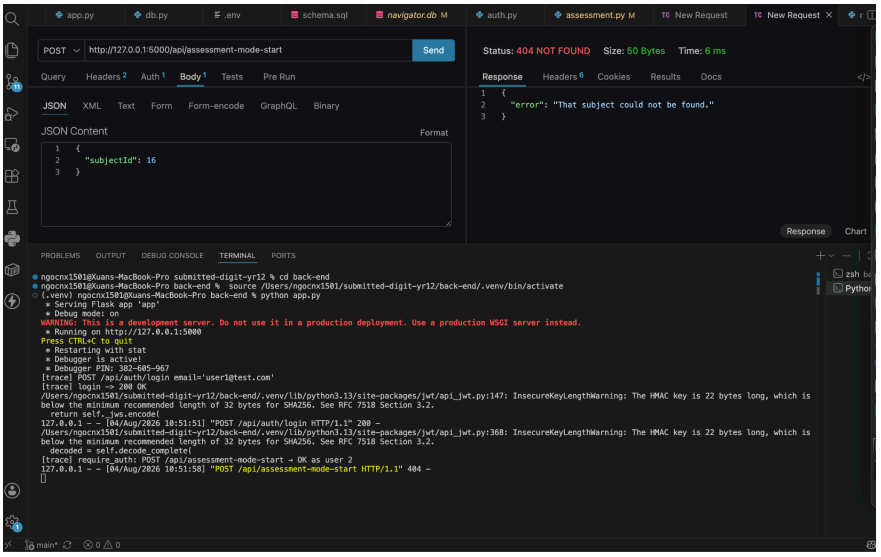
```python
"""Routes for pausing a subject during an internal assessment, then catching it up."""

from __future__ import annotations

from flask import Blueprint, g, jsonify, request

import scheduling
from auth import require_auth
from clock import today
from db import get_db
from errors import ApiError

bp = Blueprint("assessment", __name__, url_prefix="/api")


def _owned_subject(db, subject_id, user_id):
    owns = db.execute(
        "SELECT 1 FROM subjects WHERE id = ? AND user_id = ? AND archived = 0",
        (subject_id, user_id),
    ).fetchone()
    if not owns:
        raise ApiError("That subject could not be found.", status=404)


@bp.post("/assessment-mode-start")
@require_auth
def start():
    data = request.get_json(silent=True) or {}
    subject_id = data.get("subjectId")
    db = get_db()
    _owned_subject(db, subject_id, g.user_id)
    db.execute("UPDATE subjects SET internal_mode = 1 WHERE id = ?", (subject_id,))
    db.commit()
    return jsonify(ok=True)


@bp.post("/assessment-mode-end")
@require_auth
def end():
    data = request.get_json(silent=True) or {}
    subject_id = data.get("subjectId")
    db = get_db()
```

So takes the subject_id sent in the request body and the authenticated user's ID (g.user_id) from the JWT token.

It queries SQLite with a SQL statement looking for a row matching both the subject_id and the user_id.

If no matching record is found (meaning the subject doesn't exist or belongs to someone else), execution stops immediately before any database updates can occur.
With _owned_subject(db, subject_id, g.user_id) active, I re-sent the exact same request in Thunder Client (User 1's token attempting to modify User 2's subjectId: 16).

And this is the result:



I actually chose to return a 404 Not Found status instead of a 403 Forbidden. This is because returning 403 will tell an attacker "That subject exists, but you aren't allowed to see it", which leaks information about valid record IDs (ID). Returning 404 ensures that trying to access another user's data looks identical to requesting a record that doesn't exist at all.

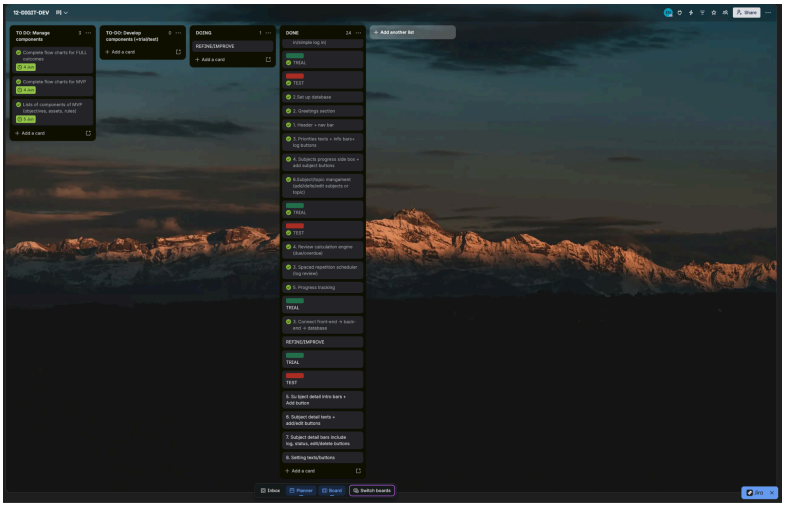
The tester navigated to the Biology subject page and scrolled down to the "Start Internal mode" button.	To know exactly what the button does before clicking it.	The tester hesitated and asked me, "Wait, if I click this, will it delete my topics or just pause them?" Because the button just said "Start", the user lacked the confidence to use it. They didn't realise it was a safe pause feature or specific things that would happen.	I do think that this problem raised by the end user is quite significant, because if other users hesitate to click the internal mode buttons then its functions would be useless. I realised the UI needed to clearly explain the state of the dashboard. I added a dynamic yellow banner that only renders when the mode is active.
The tester looked at a topic they had logged 8 times. They click the "View history (8)" dropdown to see their past reviews.	A clean, simple interface.	The tester felt the bulleted list of 8 past dates, confidence levels, and reflection notes (e.g., "I keep forgetting the Makonikov rules") was "too much text". They suggested I delete the history log feature entirely to make the page look cleaner.	I do recognise their concern about UI clutter, but I rejected their solution to delete it. Allowing students to read their past reflection notes is a core feature for identifying weak points before an exam. So I still decide to keep the view history

Project management:

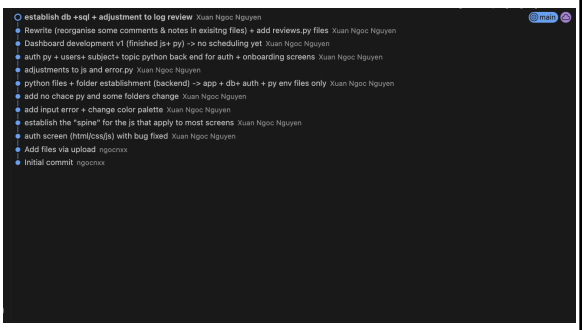
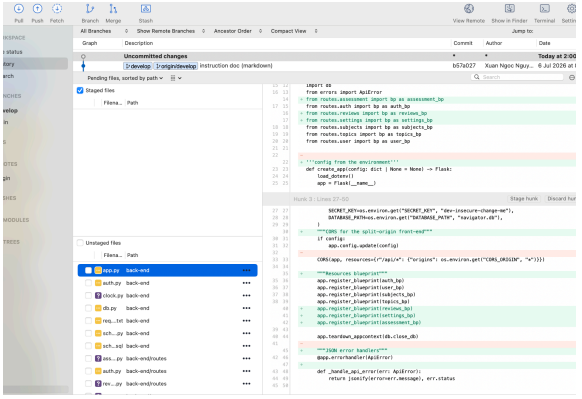
Development PROJECT MILESTONE TRACKER

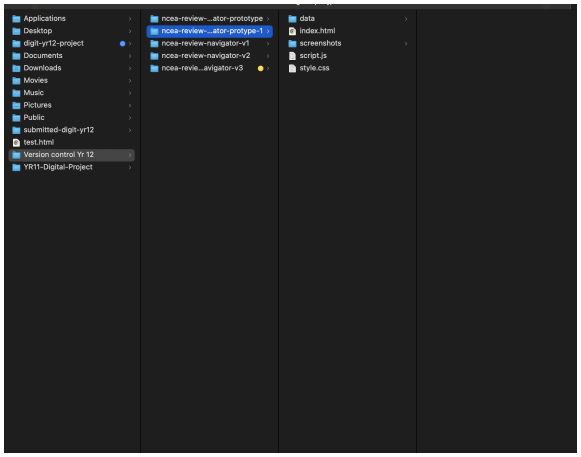
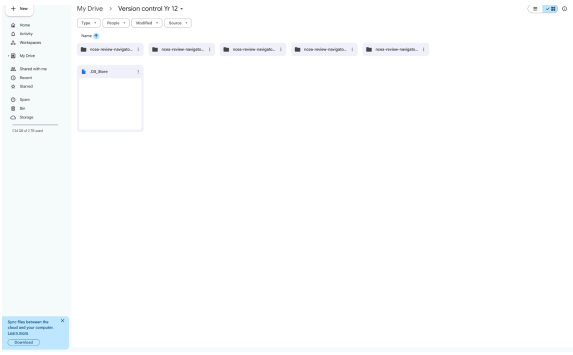
STATUS KEY: Completed | Revised/Late | Rescheduled | Upcoming

#	Milestone / Task	Deadline	Time took to complete	Time completed	Status
▶	**SETUP**				
1	Set up git + project folder	22/06/2026	1	21/06	Completed
2	Download python + technology needed	24/06	1	27/06	Completed/Late
3	Set up basic html + css+ js files	25/06	1	26/06	Completed/Late
▶	**COMPONENT DEVELOPMENT (AUTH)**				
4	Build auth screen html + css (trial throughout)	30/06	2	28/06	Completed
5	Build BASE javascripts for all screen	04/07	3	01/07	Completed
6	Build the javascripts for auth screen	07/07	2	03/07	Completed
7	Trial the functionality of front end	08/07	1	03/07	Completed
8	Build the back end + DB (users + passwords+ log in status)	13/07	4	06/07	Completed
9	Connect the components and trial + test	13/07	1	06/07	Completed
▶	**COMPONENT DEVELOPMENT (ONBOARDING)**				
10	Build onb screen html + css (trial throughout)	15/07	1	07/07	Completed
11	Build the javascripts for onb screen (and add some more in the base js files)	17/07	1	08/07	Completed
12	Trial the functionality of front end	18/07	1	08/07	Completed
	Build the back end + add subjects/topics files	21/07	4	11/07	Completed
13	Connect the components and trial + test	21/07	1	12/07	Completed
▶	**COMPONENT DEVELOPMENT (DASHBOARD)**				
14	Build dashboard screen html + css (trial throughout)	23/07	1	15/07	Completed
15	Build the javascripts for dsb screen (and add some more in the base js files)	25/07	2	16/07	Completed
	Trial the functionality of front end	25/07	1	16/07	Completed
	Build the back end + DB (add log review + add subject/topic functions), progress graph)	29/07	6	23/07	Completed
	Build the scheduling algorithm (back end AND js for visual update)	02/08	3	25/07	Completed
16	Connect the components and trial + test	02/08	1	25/07	Completed
▶	**COMPONENT DEVELOPMENT (SUBJECT DETAIL)**				
17	Build subject detail html + css (trial throughout)	04/08	2	26/07	Completed
18	Build the javascripts for subject detail screen (and add some more in the base js files)	06/08	3	28/07	Completed
19	Trial the functionality of front end	07/08	1	29/07	Completed
20	Build the back end + DB (review count, show review evidences from db)	09/08	5	02/08	Completed
21	Connect the components and trial + test	10	1	05/08	Completed
22					



Review tools and techniques	
Screenshots of planning tools	Summary
The trello + google sheet screen shots can be seen across the development in section 3 (and how it changes throughout the development). Here is my local folder to save versions: And this is my gg drive folder to save versions:	Because my web app had a lot of moving parts, I had to be really organised with my planning. I broke the project down into smaller, manageable chunks rather than trying to build it all at once. Trello (Task Tracking): Trello was my main tool for tracking what I needed to do. I used it like a Kanban board, breaking big jobs (like 'build the backend' or 'make the database') into smaller task cards. I sorted these into columns: "To-Do," "Doing, and "Done." Moving the cards across the board was really helpful because it visually showed me exactly what I was working on at any given time. This stopped me from getting overwhelmed or distracted by trying to build too many features at once. Google Sheets (Timeline and Scheduling): Trello tracked what I was doing, my Google Sheet tracked when things needed to be done. I used it as a strict timeline, logging the exact dates I finished tasks and setting deadlines for upcoming milestones. I actually had to adapt this during development. For example, when I was building the backend, fixing my database security flaw (the bola bug) took a lot longer than I originally planned. Because I was tracking my progress on the sheet, I saw I was falling behind, so I adjusted my timeline. I pushed the final CSS

	visual polishing back by a few days to make sure I had enough time to get the core security working properly without rushing.
Any edits or updates to planning tools Github:  Source Tree: 	GitHub is my main version control system. My main technique was making sure I committed my code every single time I got a feature working properly. I always wrote a short message explaining what I did (e.g. ADD INPUT ERROR _ CHANGE COLOR PALETTE seen in screen shot next to it,). This gave me a full history of my project. It was really helpful for letting me experiment freely because if I tried writing a new feature and it completely broke my site, I knew I could just roll back to my last working commit without permanently losing my hard work. I also use SourceTree alongside GitHub because it gives a really good visual breakdown of my code changes. Before I actually committed anything to GitHub, I would check SourceTree to look at the "diffs", it highlights exactly which lines of code I added, changed, or deleted. This helped me double-check that I wasn't accidentally saving broken code or typos into my main project. It also made it really easy to find old pieces of code that I had overwritten but later realised I actually needed back.

  Local + GG drive back up	I also knew that relying completely on GitHub was a bit risky because if my account got locked or I accidentally deleted a branch, I could lose everything. To prevent this, I also made manual backups on my laptop and synced them to Google Drive. I used a specific file naming system for these folders, like homepage_v1 and backend_test_v2. This gave me safe, easy-to-access snapshots of my whole project at different stages (from the first HTML prototypes to the final backend). This technique guaranteed that no matter what happened with the internet or Git, my files were always safe and organised right on my computer.

Task 4: Develop and Test Components (A-M)

In this section we are testing components of our outcome to ensure they are functional.

Final test of all components

Explanation

Before giving the web over to real students, I needed to do a final, technical test of every component in the system. So I go to terminal, activate backend + front end and went through the web module by module (Authentication, Onboarding, Dashboard, and the Review Engine) to ensure the frontend JavaScript, the Flask backend, and the SQLite database were all talking to each other correctly. Because I had been trialling things continuously during development, I expected most of this to work, but this final test is also important to ensure that combining all the components together didn't cause any unexpected bugs or security flaws (like the BOLA issue I fixed earlier).

Test case (item being tested)	What do you expect?	What happened?	Action required?
Auth: User attempts to log in with different capitalisation in their email (e.g., User@gmail.com).	The database should treat the email as case-insensitive and log them in successfully.	The system logged the user in perfectly. The COLLATE NOCASE database fix from earlier held up.	None
Onboard: Add a new ncea subject & 3 topics	Frontend should send post request, db save it and topics appear instantly on dashboard	Db did save and refresh page	None
BOLA:	When User A's token requests User B's subject ID, the server should return a 404 Not Found.	The _owned_subject() gatekeeper function intercepted the request. A 404 error was returned. User B's data remained safe.	None. It passed the test:)
Space repetition schedule: Spaced Repetition interval calculation	calculate next_due dates after log a review and update the database.	Interval updated correctly	None
Assessment mode: Toggling pause/internal mode	The backend should update the internal_mode boolean to 1, and the UI should hide the daily review notification for	The database updated instantly and the UI hid the cards, preventing the user from being overwhelmed	None

	that subject.	during an internal week.	

End-user test

Explanation

The technical trialing above only proves that the code doesn't crash but it doesn't prove that the web is actually usable. For this phase, I sat down with a few of my classmates (my target audience of stressed NCEA students) and watched them use the app on their own devices. I give them no instructions or help, just asked them to sign up, add a subject, and do a review session. I was specifically looking at their behaviour to see if the UI made sense to someone who didn't write the code.

Test case (item being tested)	What do you expect?	What happened?	Action required?
Navigating from Sign Up to the Dashboard.	The user should know exactly what to do after creating their account.	User sign up but hesitate on empty dashboard bc it wasn't obvious how to add first subject	I need to add an "empty state" placeholder on the dashboard that says, "You have no subjects yet. Click the + button to add your first NCEA subject!"
Editing a typo in a topic name.	The user clicks the edit icon, changes the text, and saves it.	The user found the edit button easily and the name updated on the screen immediately.	None. Passed.
Log review modal	The main title ("Log a Review") should be the largest text, followed by clear section headers	Testers found the typography hierarchy confusing. The main title "Log a Review" was the	I need to change the CSS font scaling. by increasing the main title to 1.5rem bold, set

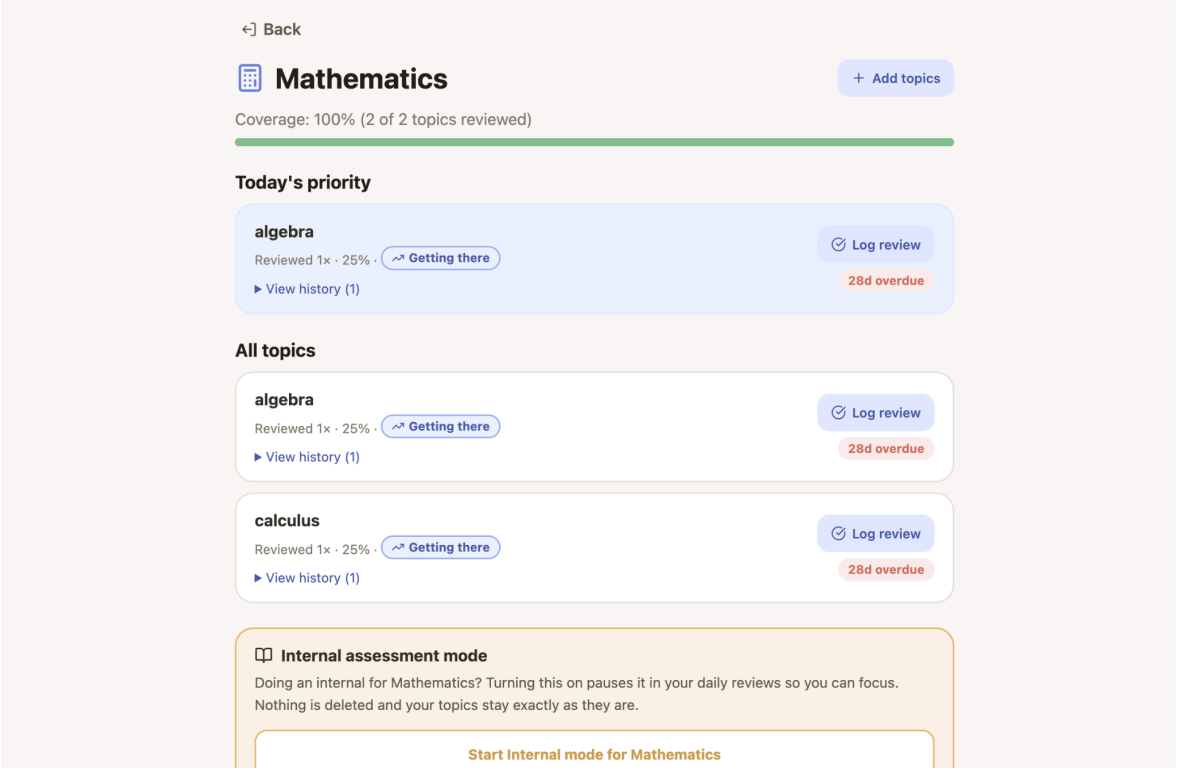
	("How well did it go?", "Reflection"), with option buttons and text boxes being smaller.	same size as the option buttons, and the rating labels (*"Need more time"*, *"Getting there"*, *"Pretty well"*) and text box subtext were actually larger than their section titles.	section titles to 1.1rem, and shrink the rating buttons and reflection subtext to 0.9rem to for a clear visual level of text.
Log review modal part.2	The log modal should feel spacious and clean, with the topic name (e.g., Photosynthesis) clearly separated from the title, and each step clearly differentiated.	Testers say that the modal feels overcrowded with too much text and info. The topic name (Photosynthesis) was squished right underneath "Log a Review" with no spacing styling, making the modal look like one giant block of text.	I will have to trim down the instructional text to reduce copy clutter. Then I will add margin-bottom: 1.25rem spacing between sections, styled the topic name in a distinct accent colour badge, and add light background cards behind each section to clearly differentiate the rating step from the reflection step.

Evidence of testing

Here you might include screenshots of how you tested your outcomes functionality

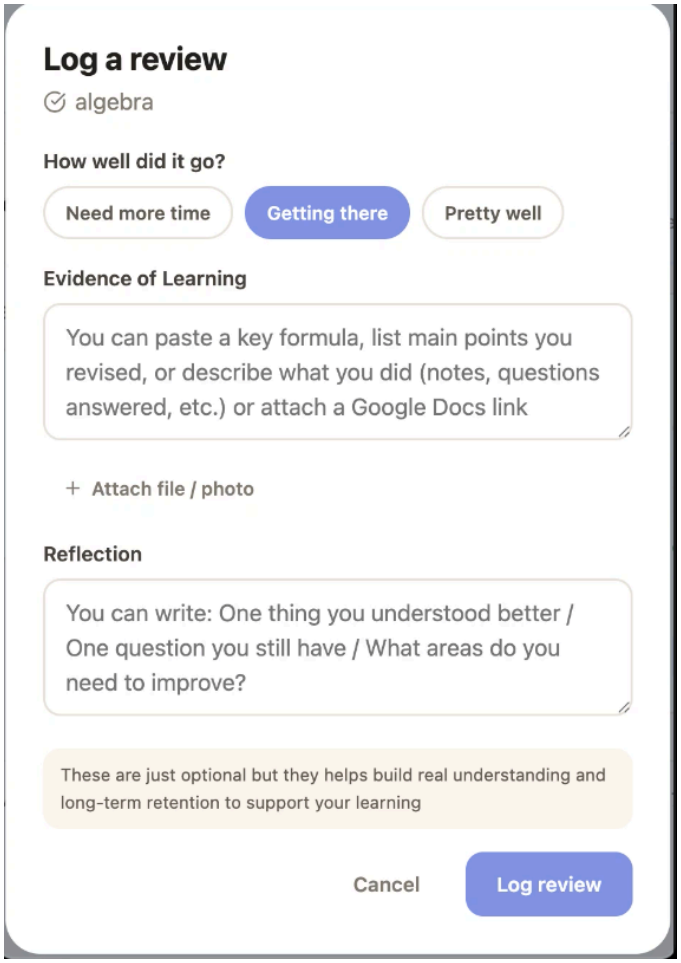
I trialled the scheduling and internal mode button functionality and the site did record all the assessment internal period + all the log review confident rate + accurate count of revision time. The internal assessment mode dynamic yellow

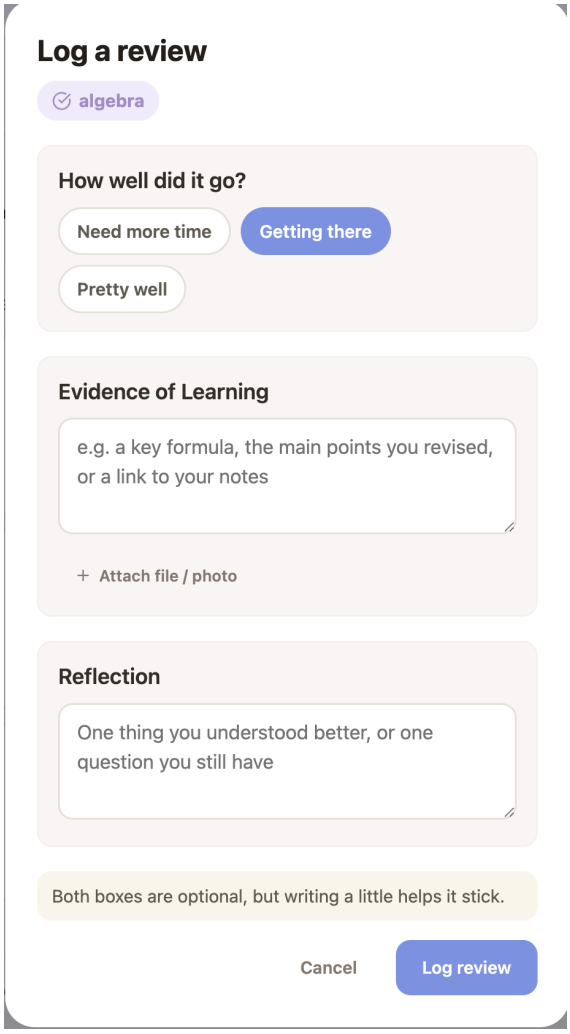
banner also works well and appears as I expected. (see Task 4 evidence)



———————

1. The log review screen that the tester see at first:

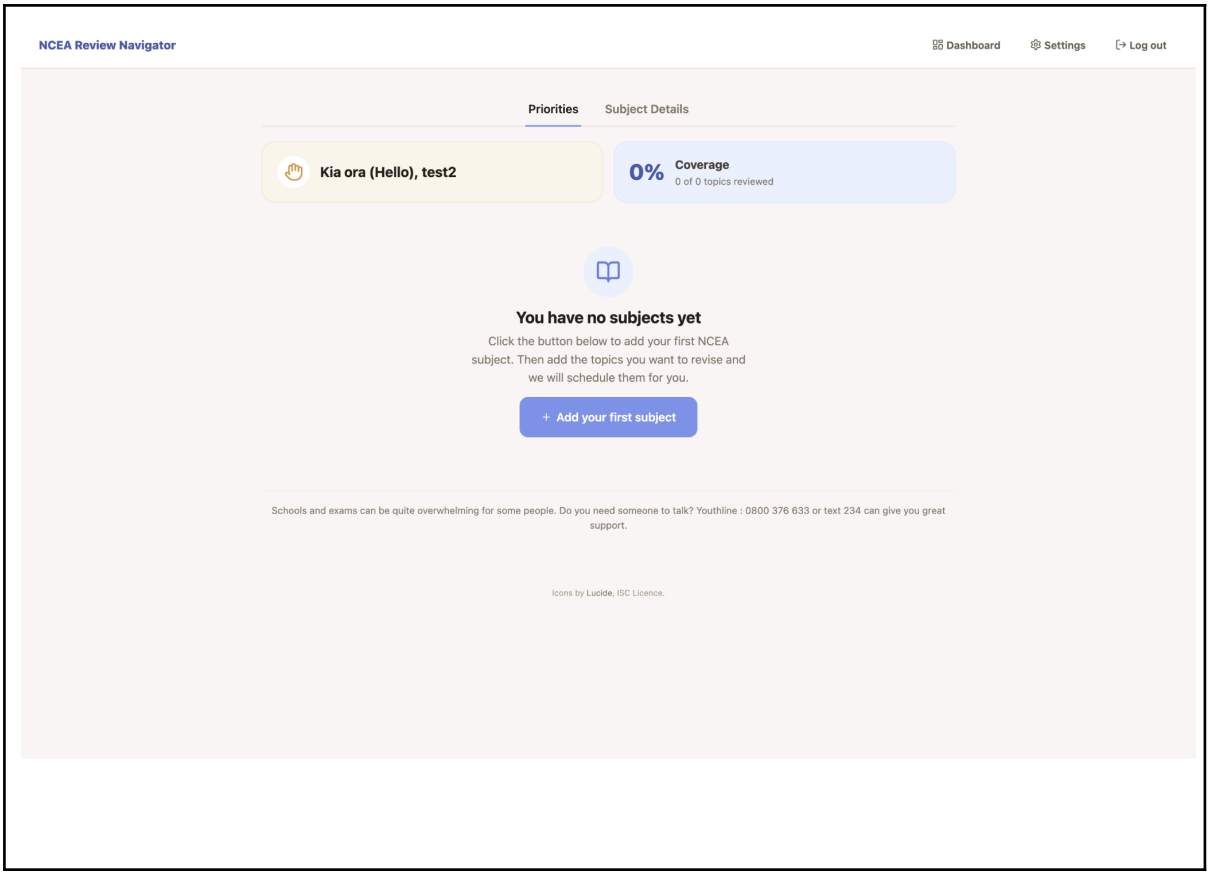

Log a review
algebra
How well did it go?
Need more time
Getting there
Pretty well
Evidence of Learning
You can paste a key formula, list main points you revised, or describe what you did (notes, questions answered, etc.) or attach a Google Docs link
+ Attach file / photo
Reflection
You can write: One thing you understood better / One question you still have / What areas do you need to improve?
These are just optional but they helps build real understanding and long-term retention to support your learning
Cancel
Log review



2. Edited log review (more organised with clear boxes and text hierarchy) which the tester now confirmed that it's much cleaner and easier to see.

—---

And here is an adjusted starter dashboard (before it was blank but now i add instruction where to start.



Explain how the testing makes sure your outcome functions as intended to meet its purpose and requirements

Testing was the connection between my initial plan and an actually successful final product. My main purpose was to help NCEA students study consistently without feeling overwhelmed, and it had technical requirements to be secure and dynamic.

The final technical testing ensured that the digital outcome met its strict functional and security requirements. As I systematically tested the JWT tokens, the API routes, and the spaced repetition math, I proved that the backend architecture was secure and functional. And I fixed the BOLA vulnerability during testing, so I could guarantee user privacy, ensuring that no student could accidentally (or maliciously) access another classmate's study schedule. Adjusting the interval multiplier during the backend test also meant the app actually suited the timeline of an NCEA school year, that fit for purpose.

However, technical testing alone wasn't enough for my web to be published/finalised. The end-user testing is what ensured the outcome was genuinely usable and accessible for my target audience. Watching real students

interact with the UI can exposed blind spots I had as the developer. For instance, when I noticed that users hesitated to click the internal mode button, it led me to add like a description of how the button works for better clarity and navigation. It made the web much more accessible and easier to understand, and prevent frustration caused by errors from mistaundarding.. As I combine both technical and user testing,it really helps meI built an secure tool that genuinely solves the problem of study overwhelm for NCEA students.

Task 5: Relevant Implications (A-M)

Implication 1: Intellectual property (IP)
Explain the implication that is relevant to your outcome.
Because I built the whole front end and back end myself using vanilla JS, Flask, and SQLite, intellectual property mainly came up around how I used other people's tutorials and docs while learning. I relied on a few outside resources to understand concepts I haven't used before, like ES modules, event delegation, and how to set up Python/SQLite projects from YouTube walkthroughs (like spaced repetition and Python timezone work). I made sure I only used these as a guide to understand the concept, not as code I just copied and pasted in.
Use evidence to show how you addressed this implication.
For example, with the spaced repetition video, I noted myself that the open-source project's code was completely different to mine due to the different context. I adjusted the logic to fit my own database structure (four separate tables instead of one big one, based on what I learned from a databases tutorial), and wrote the SQL from scratch for my specific tables (users, subjects, topics, reviews). The same applied to the SQLite tutorial content; I used it to understand syntax like COLLATE NOCASE and foreign keys, and then wrote my own schema. (Note: My full list of reference links covering these modules and tutorials can be found in my Task 3 evidence on page, which proves I properly attributed the

sources I learned from).

I also made sure all the branding, the name "NCEA Review Navigator," the logo, the UI design and layout, and the actual copy on screen (like the greeting text and error messages) are things I designed myself rather than pulling from an existing dashboard. None of the screenshots in my portfolio are stock images or someone else's UI; they're all my own build at different stages of development, which also doubles up as my version history evidence.

For the user interface icons (such as the navigation icons, edit buttons, and dashboard toggles), I deliberately avoided pulling random, uncredited images from Google Images, which could violate copyright law. Instead, I integrated Lucide Icons, a permissively licensed, open-source icon library. Lucide is licensed under the ISC license, and to ensure I fully complied with its attribution requirements, I included an appropriate credits line in my README file and my fotter.

Implication 2: Privacy
Explain the implication that is relevant to your outcome.
Privacy was a big one for me because the app stores real personal information (name, email, password) and student study habits. I wanted to make sure my database structure aligned with the core principles of the Privacy Act 2020 (keeping personal data secure and only holding it for as long as needed). I addressed this in a few concrete ways rather than just saying "I care about privacy."
Use evidence to show how you addressed this implication.
First, passwords are never stored as plain text. They are hashed before being saved into the password_hash column (which you can actually see in my navigator.db screenshots). I also use JWT tokens with a SECRET_KEY loaded securely through python-dotenv rather than hardcoded into app.py, so the key isn't sitting exposed in my source files.

Second, I kept my login error message generic ("Incorrect email or password") even after a tester told me it was confusing. I explained this decision properly in my trialling notes: if the backend tells a hacker specifically "that email doesn't exist," they could run a script against thousands of addresses and build a list of real student emails just from the error message. Protecting that list of real users was more important than a small UX inconvenience.

Third, every subject and topic table has a user_id foreign key linking it back to the account that created it, and my @require_auth guard checks that link before letting any request through. I actually tested this on purpose, logging out and trying to POST directly to /api/subjects without a token, and the backend correctly blocked it with a 401 error, printing require_auth: 401 (no token) in the terminal. That's direct evidence that unauthenticated users can't see general data.

To take it a step further, I also tested for Broken Object Level Authorization (BOLA). While the 401 error stops people without an ID, a BOLA vulnerability is when a user does have their own token but tries to access someone else's info. I discovered that by manually changing the subject ID parameter in an API request using thunder client, a logged-in user could intercept another classmate's subject details. I refactored the Flask backend routes to enforce strict user scoping using g.user_id verification (via an _owned_subject() helper function). I re-tested this fix, that co0nfirm the backend successfully returns a 404 Not Found if User A tries to query User B's data.

Finally, for data retention, I built cascading deletes into my schema (detailed in Task 3). If a user deletes their account, all their linked subjects and reviews are wiped automatically.

Implication 3: Usability and Functionality
Explain the implication that is relevant to your outcome.
I tested usability and functionality constantly throughout development, making sure that every component actually works end-to-end. Functionality is relevant to my outcome because the scheduling engine, progress tracker, and login system have to actually function without crashing in order for the end-users to rely on it.

Usability is relevant to my outcome because my target audience consists of stressed NCEA students who are already juggling heavy study workloads. If the web is confusing to navigate, visually cluttered, or hard to read, students will feel overwhelmed and quickly abandon the tool. Usability ensures that the visual hierarchy, text sizes, button placements, and interface layouts are clear and accessible across differen users, reducing cognitive friction.
Use evidence to show how you addressed this implication.
On the functionality side, I make sure core actions actually work end to end under real conditions, not just in a port. Signing up, logging in, adding a subject, log a review and seeing the dashboard update are all things I tested with a working database connection, not mock data, and I caught and fixed real crashes along the way (the 500 error from the missing users table, the None + 1 TypeError on a brand new topic's first review, the KeyError once a topic had been reviewed more than three times). Fixing all of these means the scheduling engine, which is the actual point of the webb, can work no matter how a student uses it. On the usability side, the one of the change was adding the two stages "Log saved!" confirmation (a tick in the modal, then a green toast as it closes) after a tester said she couldn't tell if her review had actually been saved, since the progress bar changing wasn't a strong enough signal on its own. I also had to change the subject picker after seeing a <datalist> filter its own suggestions based on what's typed, meaning once a student picked "English," the list collapses and they'd have to delete the word letter by letter to see other options again. So I swapped to a proper <select> with an "Other" option fixed this completely, and I checked it worked by testing switching between multiple subjects in a row rather than just picking one and stopping.

Implication 4: End user consideration
Explain the implication that is relevant to your outcome.
My end users are NCEA students, most of them under time pressure and stress around exams, so a few decisions that weren't purely technical but adjusted based on their need.

Use evidence to show how you addressed this implication.
To ensure the web was culturally appropriate for my New Zealand audience, I deliberately designed the dashboard to greet the user in te reo Māori ("Kia ora (Hello), [User]"). I kept the English translation in brackets for cognitive accessibility, but including the te reo greeting helps the interface feel local and welcoming to NZ secondary students. One of the examples is the 1 new topic per subject per day cap on the dashboard feed. A tester who added five new topics at once asked me why she could only see one, and said she would rather see all five. I decided not to change this, and explained why in my notes: giving a student five new topics on day one goes against the whole point of spaced repetition, which relies on spreading reviews out, and would recreate the exact overwhelm the web is meant to prevent. I did make the messaging clearer though, adding "+4 more topics when you're ready" so students understand it's a design choice and not a bug or missing data. I also built a clock.py module using Python's zoneinfo library to calculate "today" based on Pacific/Auckland time specifically, including daylight saving, rather than relying on the server's default UTC clock. This came directly from a tester's question about what happens if she studies late at night, and it matters because most free hosting runs on UTC, which is up to 13 hours behind New Zealand, so without this fix a student review at 11pm could have had their work logged as "yesterday" and had their whole schedule is thrown off. And I added a short support message on the dashboard itself ("Schools and exams can be quite overwhelming for some people. Do you need someone to talk to? Youthline: 0800 376 633 or text 234 can give you great support.") without being asked to. Given the target audience is stress exam students, I feel the web should acknowledge that studying isn't just a technical problem, and point students somewhere real if they need it.

Task 6: Evaluate (E)

Discuss how the information from planning, testing and trialling of components assisted in the development of a high-quality outcome.

Improving the quality of the NCEA Review Navigator was achieved by multiple iterations of development, trialing components on my own an testing them on real users, then feeding whatever i learned back into the next round of work.Each part of that loop caught things that wouldn't have been detected on their own, had I skipped any part of this trialing testing, the outcome would have been noticeably weaker.

When I look back at my initial requirements, I can confidently state that the final outcome successfully fulfills its intended purpose. The NCEA Review Navigator directly supports SDG 4 (Quality Education) by giving students a functional tool to manage their learning. The core requirement of Secure Authentication was met through the implementation of JWT tokens and the fixing of the BOLA vulnerability (using g.user_id scoping), that ensure complete privacy for the end users. The Core Functionality of the spaced-repetition engine works exactly as intended, dynamically scheduling review dates based on user input. And the addition of the 'Internal Mode' toggle successfully meets the requirement for study flexibility during busy NCEA assessment weeks. Finally, through extensive stakeholder testing and CSS refactoring (such as fixing the text hierarchy issues), the web meets its Usability requirement, resulting in a clean interface.

The planning process gave me structure and helped me know where to direct my attention. I break the outcome down into five components in Task 2, then lay them out on the Milestone Tracker and Trello board. This meant that every bug or piece of feedback had obvious places that it can be fixed to. I could see straight away what component each task belonged to, for example “auth” or scheduling, and whether fixing it properly would push my other deadlines back. And this process changed how I worked. When the bola bug fix took a lot longer than I'd planned for, I could see myself falling behind on the tracker in real time, so I made the call to push the CSS polish time back rather than rush the security side. This can ensure I finished the component up to the highest standard (not rushing) while making sure that the project was completed on time. It's the planning information that helps me to protect the quality of the outcome rather than just tracking my time.

I find that the Agile incremental development alongside a Kanban board was the best approach for this project because it stopped small bugs from accumulating

into massive integration failures at the end. By building each feature end-to-end including connecting the SQLite database, Flask backend, and frontend UI for each component before moving on, I was able to catch broken data links straight away (e.g. 404 errors or CORS errors) instead of trying to fix everything during a high-stress final assembly. The Kanban board also gave me a clear visual layout of my priority tasks, keeping me focused on core functionality like user auth and the scheduling engine without getting distracted by scope creep. Pairing this step-by-step workflow with flexible scheduling gave me the breathing room to properly debug complex backend logic without rushing, and ensuring the final project was stable, and fully functional with all components needed to function.

Then, trialing on my own helped me find problems I never would have thought to plan around in advance. The CORS and module path issue is one of the most common errors I encountered. I genuinely expected index.html to just open and work, and instead I found out browsers block local file imports for security reasons I didn't even know existed until I hit the error. Without trialling that specific failure properly, reading the console, working out what CORS actually meant, then deliberately setting up a local Python server, my whole modular JavaScript setup simply wouldn't run. This is because the browser would keep blocking every import between main.js, router.js and the screen files. The backend trials also worked the same way. As I tried a fresh sign up , I found the None + 1 crash on a topic that had never been reviewed, and trialling a fourth review on the same topic found the KeyError once my interval dictionary ran out of keys. Neither of these would show up in a quick five minute demo, they only appeared because I deliberately trialled the specific cases, a brand new topic and a heavily reviewed one, instead of just the obvious happy path. Fixing both of these with safe fallbacks is what actually makes the spaced repetition engine, which is the whole point of the app, something that keeps working properly for my end user rather than breaking the second a real student uses it for more than a week.

Lastly, I tested with actual people to close the gap between something working and something being usable, which my own trialling could never have caught by itself. My trialling told me the password validation, login and dashboard all functioned correctly. It took a real tester physically missing the small red text at the bottom of the form to show me that functionality isn't the same as usability, and that feedback is directly why the red input border exists now. In the same way, a classmate typing Chloe@gmail.com at sign up and then chloe@gmail.com at login exposed a case sensitivity bug in SQLite I would never have caught testing with my own consistent typing. The feedback helped me to fix it with COLLATE NOCASE, that made the login behave the way real students, typing carelessly on their phones, actually

expect it to. The Log Review saved confirmation exists for the exact same reason, my trialling confirmed the request worked on the backend, but only a real user showed me that silence after clicking a button reads as failure, not success.

Just as important as taking feedback on board was knowing when to justify going against it. When two testers hit the identical Incorrect email or password message and called it confusing, I chose not to split it into two separate messages, because doing that would let someone run a script to work out which student emails are real, and I decided that privacy risk mattered more than a small UX annoyance. In the same way, when a tester asked why she could only see one new topic a day instead of all five she'd added, I kept the limit exactly as it was, because giving a student five brand new topics on day one goes against the whole spaced repetition idea the web is build around and would just recreate the overwhelm it's meant to stop. I only changed the wording around it, adding the "+4 more topics when you're ready" message, so the limit is more like an intentional choice. These two moments really matter to my development time, because they help me to actually weigh testing information against other priorities, security and the learning theory behind the web. So I can see multiple opinions and dimensions of the web that I've missed and make choices from it.

Version control is what makes this entire project survive in the first place. Because every working feature was committed to GitHub, and checked against the diff in SourceTree before I committed it, I was willing to trial riskier changes, like ripping out the broken datalist subject picker and rebuilding it as a proper select menu with a hidden custom input, without worrying I'd lose a version that already worked. Keeping local and Google Drive backups on top of that also meant a GitHub problem wouldn't have cost me the whole project. That safe place is honestly why I kept re-trialling things like the scheduling algorithm three separate times instead of settling for the first version that technically ran without crashing.

Research also included informed decisions I made, not just trialling and feedback. Watching the Databases 101 video before writing schema.sql is a good example. Splitting the data into four tables linked by foreign keys is why deleting a user automatically cascades to clean up their subjects, topics, and reviews without extra backend logic. It also keeps text constraints like COLLATE NOCASE centralised on the primary users table instead of repeating string comparisons across references. In the same way, watching the open-source spaced-repetition tutorial walkthrough didn't really give me code I could copy, since the context was completely different, but it showed me how to think about intervals as a dictionary lookup with a fallback

default LIKE the exact pattern I adapted into INTERVALS and INTERVAL_MAX. I combined research from different sources with feedback from users and my own trialling,which resulted in the database structure, the interval logic, and the timezone handling that were all built to suit users' needs.

Put all together, I don't think any single stage on its own could have got me to this outcome. Planning without trialling would have delivered an architecture that failed the second a real user hit an edge case, a brand new topic, a capitalised email, a fourth review. Trialling without testing would have delivered something technically correct that real students still found confusing or didn't trust to use it. So they may abandon it. And testing without actually being willing to push back on feedback would have produced something easy to use but less secure or less effective as an actual study tool. It's the continuous loop between planning, trialling, testing and then deciding, sometimes to change something and sometimes to defend why I wasn't going to, that turned five working components into something that actually does what it's meant to, help stressed NCEA students review consistently without feeling overwhelmed by it.

Task 7: Present your Outcome

Link to github (already included the videos):
https://github.com/ngocnxx/submitted-digit-yr12

Link to how to access the web via codespace github (create new code space -> open readme):
digit-12-instruction-codespace.mov

Here is the video demo:
final-digi-demo

Link to all the video files:
https://drive.google.com/drive/folders/1OTPIoH9HEnFxVV4s93MoaT8UkhXLqfa_?usp=drive_link